

Quantum

Quantum-Secure Privacy Blockchain

Technical Specification v1.0

Draft – January 2026

Phexora AI

<https://quantum.phexora.ai>

This specification is released under CC0 (Public Domain).
Designed for AI-verifiable implementation.

Abstract

Quantum is a research specification for a privacy-by-default, high-throughput blockchain designed to remain secure against both classical and quantum adversaries. It addresses three fundamental limitations of existing blockchains: vulnerability to quantum computers, lack of default privacy, and limited scalability.

The core research challenge: No existing blockchain combines DAG-based consensus (like Kaspas's GhostDAG) with full transaction privacy. Quantum aims to solve this, enabling 1,000+ TPS on L1 while maintaining complete anonymity.

This specification combines five key innovations: **(1)** DAG-based consensus (GhostDAG) enabling parallel block creation and 10-32 blocks per second, **(2)** lattice-based commitment schemes (Module-LWE) that hide transaction amounts while preserving verifiable supply integrity, **(3)** hash-based signatures (SPHINCS+) and key encapsulation (ML-KEM) that resist quantum attacks, **(4)** transparent zero-knowledge proofs (STARKs) that require no trusted setup ceremony, and **(5)** privacy-preserving proofs over DAG structures—the novel cryptographic contribution that enables private transactions in a parallel-block architecture. The result is a system targeting Kaspas-level throughput with Monero-level privacy and post-quantum security.

This document serves as both a formal specification and an implementation guide. It is explicitly designed to be machine-verifiable, enabling AI systems to implement, test, and formally verify conformant implementations. All cryptographic parameters are fully specified, all algorithms are deterministic, and all ambiguities are resolved in favor of security.

Contents

Abstract	i
Introduction	1
The Quantum Threat to Blockchain Security	1
The Privacy Imperative	1
Our Approach	1
Design Philosophy	2
Why Post-Quantum Now?	2
Why Lattice-Based Commitments Over Pedersen?	2
Why STARKs Over SNARKs?	3
Why Privacy-by-Default With No Opt-Out?	3
Why DAG-Based Consensus (GhostDAG)?	3
Mining Algorithm (Open Research Question)	4
Why These Specific Parameters?	4
Why Bitcoin's Foundation? UTXO Model, Nakamoto Consensus, Fair Launch	5

Project Status and Research Phases	6
Honest Assessment	6
Research Phases and Milestones	6
Phase 1: Specification and Formal Analysis (Current)	6
Phase 2: Cryptographic Proof-of-Concept	7
Phase 3: Testnet Implementation	7
Phase 4: Mainnet Launch	8
What This Means for Stakeholders	8
For Researchers	8
For Investors	8
For the Community	9
Current Status	9
Quantum: Quantum-Secure Privacy Blockchain	9
Formal Specification v1.0	9
IMMUTABLE REQUIREMENTS	9
THIS SECTION IS IMMUTABLE	10
R1. PRIVACY REQUIREMENTS	10
R1.1 Privacy by Default [MANDATORY]	10
R1.2 Sender Privacy [MANDATORY]	10
R1.3 Receiver Privacy [MANDATORY]	10
R1.4 Amount Privacy [MANDATORY]	10
R1.5 Network Privacy [MANDATORY]	10
R2. SECURITY REQUIREMENTS	11
R2.1 Quantum Security [MANDATORY]	11
R2.2 No Trusted Setup [MANDATORY]	11
R2.3 Cryptographic Binding [MANDATORY]	11
R2.4 Cryptographic Hiding [MANDATORY]	11
R2.5 Proof Soundness [MANDATORY]	11
R2.6 Proof Zero-Knowledge [MANDATORY]	11
R3. DECENTRALIZATION REQUIREMENTS	12
R3.1 Permissionless Participation [MANDATORY]	12
R3.2 No Privileged Parties [MANDATORY]	12
R3.3 Permissionless Mining [MANDATORY]	12
R3.4 Open Source [MANDATORY]	12
R4. INTEGRITY REQUIREMENTS	12
R4.1 Fixed Supply [MANDATORY]	12
R4.2 No Inflation Bugs [MANDATORY]	13
R4.3 Double-Spend Prevention [MANDATORY]	13
R4.4 Transaction Finality [MANDATORY]	13
R5. FUNCTIONAL REQUIREMENTS	13
R5.1 Basic Transaction Support [MANDATORY]	13
R5.2 Wallet Functionality [MANDATORY]	13
R5.3 Light Client Support [MANDATORY]	13

R6. PERFORMANCE REQUIREMENTS	14
R6.1 Transaction Processing [MANDATORY]	14
R6.2 Storage [MANDATORY]	14
R6.3 Network [MANDATORY]	14
R7. NON-REQUIREMENTS (Explicitly Excluded)	14
R7.1 NOT Required	14
R7.2 NOT Permitted	15
R8. SCALABILITY REQUIREMENTS	15
R8.1 DAG-Native Architecture [MANDATORY]	15
R8.2 L1 Throughput Targets [MANDATORY]	15
R8.3 Parallel Processing [MANDATORY]	15
R8.4 Privacy-Preserving DAG Consensus [MANDATORY]	16
R8.5 Horizontal Scaling [MANDATORY]	16
R8.6 Layer 2 Compatibility [MANDATORY]	16
Requirement Compliance Matrix	16
Immutability Declaration	17
END OF IMMUTABLE REQUIREMENTS	18
Part I: Cryptographic Foundation	18
1. Notation and Conventions	18
1.1 Mathematical Notation	18
1.2 Security Parameter	18
1.3 Endianness and Encoding	19
2. Hash Functions	19
2.1 Primary Hash Function: SHAKE256	19
2.2 Defined Hash Instances	19
2.3 Hash-to-Field	20
3. Lattice-Based Commitments	20
3.1 Module-LWE Parameters	20
3.2 Polynomial Operations	20
3.3 Commitment Scheme	21
3.4 Randomness Generation	22
4. Hash-Based Signatures: SPHINCS+-256f	22
4.1 Parameters	22
4.2 API	23
4.3 Security	23
5. Key Encapsulation: ML-KEM-1024 (Kyber)	23
5.1 Parameters	23
5.2 API	23
6. Zero-Knowledge Proofs: STARKs	24
6.1 Overview	24
6.2 Arithmetic Intermediate Representation (AIR)	24
6.3 Field Selection	24
6.4 FRI Parameters (Fast Reed-Solomon IOP)	25

6.5 STARK Proof Structure	25
7. Merkle Trees (Quantum-Secure)	25
7.1 Construction	25
7.2 Tree Parameters	26
7.3 Append-Only Tree	26
Part II: Protocol Specification	26
8. Account and Address System	27
8.1 Key Hierarchy	27
8.2 Address Format	28
8.3 Stealth Addresses	28
9. Transaction Structure	28
9.1 Output (Note)	28
9.2 Nullifier	29
9.3 Transaction	30
9.4 Transaction Size Estimate	30
10. Validity Proof (STARK Circuit)	31
10.1 Statement to Prove	31
10.2 AIR Constraints (Detailed)	32
10.3 Proof Generation	32
10.4 Proof Verification	33
11. Consensus: Proof of Work	33
11.1 Hash Function	33
11.2 Block Header (DAG)	34
11.3 Difficulty Adjustment (DAG)	34
11.4 Block Structure	35
11.5 Block Validation (DAG)	35
12. Serialization	36
12.1 Canonical Encoding	36
12.2 Transaction Serialization	36
13. Network Protocol	37
13.1 Transport Layer	37
13.2 Message Types	37
13.3 Dandelion++ Parameters	38
14. State Management	38
14.1 Chain State	38
14.2 Database Schema	38
15. Wallet Operations	39
15.1 Key Generation	39
15.2 Scanning for Outputs	39
15.3 Creating Transactions	40
16. Economic Parameters	40
16.1 Supply Schedule	40
16.2 Fee Structure	41
16.3 Unit Definitions	41

Part III: Verification Criteria	42
17. Correctness Properties	42
17.1 Cryptographic Correctness	42
17.2 Protocol Correctness	42
17.3 Privacy Properties	43
18. Test Vectors	43
18.1 Hash Function Test Vectors	43
18.2 Commitment Test Vectors	44
18.3 Transaction Test Vectors	44
18.4 Consensus Test Vectors	45
19. Implementation Requirements	45
19.1 Mandatory Features	45
19.2 Performance Targets	46
19.3 Security Requirements	46
19.4 Code Quality Requirements	46
20. Formal Verification Targets	46
20.1 Properties to Formally Verify	46
20.2 Verification Tools	47
20.3 Audit Checklist	47
Part IV: Appendices	47
A. Reference Implementations	47
A.1 Polynomial Multiplication (NTT)	47
A.2 Lattice Commitment	48
A.3 STARK Prover Outline	49
B. Genesis Block	50
B.1 Genesis Parameters	50
B.2 Genesis Block Structure	51
B.3 Genesis Block Hash	51
C. Network Magic Numbers	51
D. Recommended Libraries	52
D.1 Rust Ecosystem	52
D.2 Alternative Language Implementations	52
E. Glossary	52
F. Document Metadata	53
G. Known Limitations and Trade-offs	53
G.1 Transaction Size	53
G.2 Proof Generation Time	54
G.3 Address Size	54
G.4 No Smart Contracts	55
H. Research Challenges and Future Directions	55
H.1 Core Research Challenge: Privacy-Preserving DAG Consensus	55
H.2 Alternative Scaling Approaches Considered	57
H.3 Mining Algorithm Analysis (Open Research)	59
H.4 Proof System Improvements	60

H.5 Layer 2 Scaling	61
H.6 Post-Quantum Signature Alternatives	61
End of Specification	62

Introduction

The Quantum Threat to Blockchain Security

The security of virtually all deployed blockchain systems rests on a single assumption: the computational hardness of the discrete logarithm problem in elliptic curve groups. Bitcoin, Ethereum, and nearly every major cryptocurrency use ECDSA or EdDSA signatures and ECDH key exchange—all of which would be broken by a sufficiently powerful quantum computer running Shor’s algorithm.

This is not a distant hypothetical. Quantum computers capable of breaking 256-bit elliptic curve cryptography could emerge within the next 10-20 years. More concerning is the “harvest now, decrypt later” threat: adversaries can record encrypted blockchain data today, store it, and decrypt it once quantum capabilities mature. For a blockchain designed to be permanent and immutable, this means that cryptographic choices made today have consequences that extend decades into the future.

The Privacy Imperative

Beyond quantum security, current blockchain systems suffer from a fundamental privacy deficiency: transaction transparency. While often marketed as a feature (auditability, trustlessness), transparent transactions create severe problems:

- **Fungibility:** When transaction history is visible, individual coins can be “tainted” by their history, breaking the interchangeability essential to functioning money
- **Commercial confidentiality:** Businesses cannot use transparent blockchains without exposing their operations to competitors
- **Personal safety:** Visible balances and transaction patterns create physical security risks for users
- **Surveillance:** Transaction graphs enable mass surveillance without user consent

Privacy is not a luxury feature—it is a prerequisite for a system that claims to be censorship-resistant. Without privacy, any sufficiently motivated adversary can identify and target users.

Our Approach

Quantum addresses both challenges simultaneously through careful selection of cryptographic primitives:

Component	Primitive	Quantum Security	Why This Choice
Commitments	Module-LWE Lattice	Yes	Binding + hiding + homomorphic under quantum-hard assumptions
Signatures	SPHINCS+	Yes	Stateless hash-based, NIST standardized

Component	Primitive	Quantum Security	Why This Choice
Key Exchange	ML-KEM (Kyber)	Yes	Lattice-based KEM, NIST standardized
ZK Proofs	STARKs	Yes	Hash-based, no trusted setup
Hashing	SHAKE256	Yes	128-bit post-quantum security with domain separation

The system enforces privacy-by-default with no opt-out mechanism. This is a deliberate design choice: optional privacy creates a smaller anonymity set and marks private transactions as “suspicious.” When all transactions are private, privacy is the norm rather than the exception.

Design Philosophy

This section explains the rationale behind key design decisions. Understanding *why* the specification makes certain choices is essential for implementers and auditors.

Why Post-Quantum Now?

The Risk: Cryptographically relevant quantum computers (CRQC) may be 10-20 years away, but blockchain data is permanent. An address created today may hold value for decades. If the underlying cryptography is broken, all historical transactions become vulnerable.

Harvest-Now-Decrypt-Later: Nation-state adversaries are already collecting encrypted traffic for future decryption. Blockchain transactions are public by design, making them trivially available for future cryptanalysis.

Migration is Hard: Upgrading cryptographic primitives in a decentralized system requires coordination across millions of users and thousands of implementations. It is far easier to build quantum-secure from the beginning than to migrate later under adversarial conditions.

NIST Standardization: The post-quantum primitives used in Quantum (ML-KEM, SPHINCS+) have completed NIST standardization. They are no longer experimental but rather ready for production use.

Why Lattice-Based Commitments Over Pedersen?

Traditional privacy coins (Monero, Zcash) use Pedersen commitments based on elliptic curve discrete logarithm. These are elegant and efficient but broken by Shor’s algorithm.

Lattice commitments based on Module-LWE provide: - **Quantum resistance:** Security reduces to the hardness of finding short vectors in lattices, which resists known quantum attacks - **Homomor-**

phic property: Like Pedersen commitments, lattice commitments can be added together, enabling balance verification without revealing values - **Binding and hiding:** Computationally binding (cannot open to two values) and computationally hiding (reveals nothing about the committed value)

The trade-off is size: lattice commitments are larger (~3KB vs ~32 bytes). This is acceptable given the security requirements.

Why STARKs Over SNARKs?

SNARKs (used by Zcash) offer smaller proofs but require a *trusted setup*—a ceremony where participants generate parameters and must destroy their secret inputs. If any participant is compromised, they could create undetectable counterfeit coins.

STARKs require no trusted setup: - **Transparency:** All parameters are publicly derived from hash functions - **No toxic waste:** No secret information that could compromise the system - **Post-quantum security:** Security relies only on collision-resistant hash functions - **Scalability:** Prover time scales quasi-linearly with computation size

The trade-off is proof size: STARK proofs are larger (~100KB vs ~1KB for SNARKs). This specification accepts this trade-off because: 1. Trusted setup risk is unacceptable for a system designed to outlast its creators 2. Proof size can be reduced through future optimizations (recursive STARKs, proof aggregation) 3. Storage and bandwidth costs decrease over time

Why Privacy-by-Default With No Opt-Out?

Many privacy coins offer “selective transparency” or optional privacy. This is a mistake:

Anonymity sets shrink: If 10% of users opt into privacy, only that 10% provides cover for each other. If 100% use privacy, everyone benefits from the full user base as their anonymity set.

Privacy becomes suspicious: When privacy is optional, choosing it signals that you have “something to hide.” This invites enhanced scrutiny on private transactions.

Network effects: Privacy is a collective good. Individual opt-out degrades privacy for everyone else.

Quantum makes privacy mandatory and identical for all transactions. There is no “transparent mode” to implement, no configuration to accidentally disable privacy, and no way for users to harm the privacy of others.

Why DAG-Based Consensus (GhostDAG)?

Traditional blockchains process blocks sequentially, limiting throughput to ~10 TPS regardless of network capacity. DAG-based consensus (Directed Acyclic Graph) allows parallel block creation, fundamentally changing the scalability equation.

GhostDAG advantages: - **Parallel blocks:** Multiple miners create valid blocks simultaneously; none are orphaned - **No wasted work:** All valid proof-of-work contributes to consensus weight - **Fast**

finality: High-confidence confirmation in under 10 seconds, practical for payment applications -

Linear scaling: Throughput increases with block rate, limited only by propagation delay

The research challenge: No existing DAG blockchain provides privacy. Kaspero achieves 10+ blocks/second but transactions are transparent. Quantum's core research contribution is solving **privacy-preserving consensus over parallel block structures**:

1. **Nullifier ordering:** Preventing double-spend when blocks are created in parallel
2. **Anonymity set coherence:** Maintaining full output set as anonymity set across DAG branches
3. **STARK proofs over DAG:** Proving transaction validity with membership in a non-linear history
4. **Taint resistance:** Ensuring DAG structure doesn't enable transaction graph analysis

This is novel cryptographic research. The specification defines the requirements; the implementation must solve these open problems.

Mining Algorithm (Open Research Question)

Permissionless mining is essential for censorship resistance. The mining algorithm must ensure: - Anyone can participate without permission - No single entity can dominate hash rate - Higher total hashrate = more security (costly to attack)

The Challenge: DAG consensus with 10-32 blocks/second requires fast hash verification.

Algorithm	Hardware	Verification Speed	Hashrate Security	DAG Suitability
RandomX	CPU	~2-3ms/hash	Lower total hashrate	Uncertain (designed for 2-min blocks)
kHeavyHash	GPU/ASIC	<0.1ms/hash	High (Kaspero proven)	Proven (Kaspero)
SHA-256d	ASIC	Very fast	Highest (Bitcoin)	Fast, proven secure

Current Position: This is an open research question. The algorithm must balance: 1. **Permissionless:** Anyone can mine without approval 2. **Speed:** Fast enough for high block rate validation 3. **Security:** High total hashrate makes 51% attacks costly 4. **Proven:** Battle-tested in production

Note: ASIC mining does not inherently harm decentralization. Bitcoin and Kaspero demonstrate that ASIC/GPU mining can remain permissionless and geographically distributed. Higher hashrate provides stronger security guarantees.

See Appendix H.3 for detailed analysis.

Why These Specific Parameters?

Parameter	Value	Rationale
Security level	≥ 128 -bit post-quantum target	Hashes at 128-bit PQ; ML-KEM-1024 & SPHINCS+-256f use NIST Level 5 parameters; STARK soundness 2^{-100}
Field (STARKs)	Goldilocks ($2^{64} - 2^{32} + 1$)	Efficient 64-bit arithmetic, 2^{32} roots of unity
Ring modulus	$q = 8380417$	Matches CRYSTALS-Dilithium, enables NTT
Polynomial degree	$n = 256$	Standard lattice parameter, efficient NTT
Block rate	10-32 blocks/second	DAG consensus enables high parallelism
Confirmation time	< 10 seconds	Sub-second block time + DAG finality
Target throughput	1,000+ TPS	Competitive with Kaspas, plus privacy
Supply cap	21,000,000	Familiar, deflationary, no tail emission debate

Why Bitcoin's Foundation? UTXO Model, Nakamoto Consensus, Fair Launch

Quantum builds on Bitcoin's battle-tested foundations rather than reinventing proven concepts.

UTXO Model: Bitcoin's Unspent Transaction Output model is superior for privacy: - Transactions consume and create discrete outputs rather than modifying account balances - No address reuse is natural (one-time stealth addresses extend this) - Parallel transaction validation (outputs are independent) - Simpler SPV proofs and state verification - Proven secure for 15+ years with trillions of dollars at stake

Account-based models (Ethereum) leak information through balance changes and require complex state management. UTXO is the natural foundation for privacy.

Nakamoto Consensus: The longest-chain (most cumulative proof-of-work) rule provides: - Objective fork choice without trusted coordinators - Security proportional to honest hashrate (51% threshold) - Permissionless participation—anyone can mine - No stake-based plutocracy or validator cartels - Proven game-theoretic security since 2009

GhostDAG extends Nakamoto consensus to parallel blocks while preserving these properties. All valid proof-of-work contributes to consensus weight; the PHANTOM ordering provides deterministic transaction ordering.

Fair Launch / No Premine: Quantum follows Bitcoin's fair distribution model: - **No premine:** Zero coins allocated before public mining begins - **No ICO/IEO:** No token sale to insiders or investors - **No founder's reward:** No perpetual tax on mining rewards - **No venture capital allocation:** No preferential access for investors - **Day-one mining:** Anyone can participate from genesis block

This ensures that Quantum's value, if any emerges, accrues to those who secure the network through proof-of-work rather than to early insiders. Fair launch is essential for credibility as a decentralized, censorship-resistant system.

Why This Matters: Projects that launch with premines, founder rewards, or investor allocations create conflicts of interest that undermine decentralization claims. Bitcoin's fair launch is a key reason it remains the most credibly neutral cryptocurrency. Quantum inherits this principle.

Project Status and Research Phases

Honest Assessment

This specification describes a system that **does not yet exist**. The core innovation—privacy-preserving proofs over DAG-based consensus—is an open research problem. No existing blockchain has solved this.

What is proven (low risk):

- Post-quantum cryptography: SPHINCS+ and ML-KEM are NIST-standardized; lattice commitments rely on standard lattice assumptions but are not standardized
- STARKs: Production-ready (StarkNet, StarkEx), no trusted setup
- GhostDAG: Production-ready (Kaspa), achieves 10+ blocks/second
- Privacy transactions: Production-ready (Monero, Zcash) on sequential chains

What is unproven (research required):

- Privacy-preserving nullifier schemes for parallel blocks
- STARK proofs over DAG membership (not Merkle trees)
- Anonymity set coherence across DAG branches
- Transaction graph privacy in DAG structure
- Mining algorithm optimization for privacy-DAG (kHeavyHash leading candidate)

What could fail:

- Proof sizes may be impractical (>1MB per transaction)
- Privacy analysis may reveal fundamental DAG information leakage
- Performance may not meet targets despite theoretical feasibility
- Unknown unknowns in the interaction of components

Research Phases and Milestones

This project follows a phased approach with explicit go/no-go decision points.

Phase 1: Specification and Formal Analysis (Current)

Deliverables:

- Complete specification (this document)
- Formal security model for privacy in DAG consensus
- Academic paper: "Privacy-Preserving Proofs Over Directed Acyclic Graphs"
- Peer review by independent cryptographers

Success Criteria:

- No fundamental flaws identified in security model

- Peer reviewers agree the approach is theoretically sound
- At least one viable solution identified for each open problem (H.1)

Failure Mode:

- Fundamental incompatibility discovered between DAG and privacy
- Action: Publish findings, pivot to sequential chain, or sunset project

Phase 2: Cryptographic Proof-of-Concept

Deliverables:

- Reference implementation of DAG-aware nullifier scheme
- STARK circuits for DAG membership proofs
- Benchmark suite for proof generation/verification
- Privacy analysis of DAG transaction graph

Success Criteria:

- Nullifier scheme prevents double-spend in parallel blocks (proven)
- STARK proof size < 500 KB per transaction
- Proof generation < 60 seconds on reference hardware
- No privacy leaks identified in DAG structure analysis

Failure Mode:

- Proof sizes exceed 1 MB (impractical for users)
- Proof generation exceeds 5 minutes (unusable UX)
- Privacy analysis reveals unavoidable information leakage
- Action: Publish findings, evaluate if trade-offs acceptable

Phase 3: Testnet Implementation

Deliverables:

- Full node implementation (Rust reference)
- Wallet implementation with proof generation
- Block explorer (privacy-preserving)
- Public testnet with community participation

Success Criteria:

- Sustained 1,000+ TPS on testnet
- < 10 second confirmation time
- Independent security audit passed
- No critical vulnerabilities in 6-month testnet operation

Failure Mode:

- Performance targets not met at scale
- Security audit reveals critical flaws
- Action: Fix issues and re-audit, or downgrade targets

Phase 4: Mainnet Launch

Prerequisites:

- All Phase 3 success criteria met
- Two independent security audits passed
- Formal verification of critical components
- Community governance established

Launch Criteria:

- Audit firms sign off on production readiness
- Bug bounty program running for 3+ months
- Economic model validated

What This Means for Stakeholders

For Researchers

This is a **genuine research project** with hard, unsolved problems:

- Novel contributions possible in each open problem area (H.1)
- Publication opportunities in top venues
- Clear problem statements with measurable success criteria
- Honest acknowledgment of what we don't know

We welcome collaboration. The specification is public domain (CC0).

For Investors

This is a **high-risk, high-reward research bet**:

Risk factors:

- Core research may not succeed
- Timeline is research-dependent, not calendar-driven
- No guarantee of mainnet launch

Mitigating factors:

- Clear milestones with go/no-go decisions
- Honest communication about progress
- Fallback options at each phase
- Novel positioning if successful

Upside:

- First mover in privacy + high-throughput L1
- Patent-free, open technology (CC0 license)
- Attracts top cryptographic talent

For the Community

We promise:

1. **Honesty:** We will communicate clearly about what works and what doesn't
2. **No vaporware:** We will not hype features that don't exist
3. **Open research:** All findings published, including failures
4. **Milestones:** Clear checkpoints where we evaluate progress
5. **Integrity:** If the research fails, we will say so rather than ship broken privacy

Current Status

Phase 1: Specification and Formal Analysis

Status: IN PROGRESS

Completed:

- Core specification (this document)
- Requirements definition (R1-R8)
- Design philosophy documented
- Open problems identified (H.1)

In Progress:

- Formal security model
- Academic paper draft
- Peer review outreach

Next Milestone:

- Complete formal analysis
- Submit paper for peer review

Quantum: Quantum-Secure Privacy Blockchain

Formal Specification v1.0

Purpose: This document serves as a complete, formally verifiable specification for a quantum-secure, privacy-by-default blockchain. It is designed to be implementable and verifiable by advanced AI systems.

IMMUTABLE REQUIREMENTS

⚠ THIS SECTION IS IMMUTABLE ⚠

The following requirements define the core properties of the Quantum blockchain. These requirements:

- **MUST NOT** be modified, weakened, or removed
- **MUST NOT** be circumvented through implementation choices
- **MUST** be satisfied by any conformant implementation
- **ARE** the acceptance criteria for the final system

Any implementation that violates these requirements is **non-conformant** and **invalid**.

R1. PRIVACY REQUIREMENTS

R1.1 Privacy by Default [MANDATORY]

Every transaction **MUST** be private.

There **MUST NOT** exist any transparent transaction mode.

There **MUST NOT** exist any option to disable privacy.

There **MUST NOT** exist any mechanism to selectively reveal transaction data without explicit action by the key holder.

R1.2 Sender Privacy [MANDATORY]

Given a valid transaction, no adversary without access to private keys **SHALL** be able to determine which outputs were spent with probability greater than $1/N$, where N is the total number of outputs in the system.

R1.3 Receiver Privacy [MANDATORY]

Given a valid transaction, no adversary without access to the recipient's view key **SHALL** be able to link any output to any address.

R1.4 Amount Privacy [MANDATORY]

Given a valid transaction, no adversary without access to private keys **SHALL** be able to determine the value of any input or output.

R1.5 Network Privacy [MANDATORY]

The network layer **MUST** implement transaction propagation mechanisms that prevent correlation between transaction origin and IP address. Dandelion++ or equivalent privacy-preserving propagation is **REQUIRED**.

R2. SECURITY REQUIREMENTS

R2.1 Quantum Security [MANDATORY]

ALL cryptographic primitives MUST be secure against quantum computers.

Specifically:

- Commitment scheme: MUST be based on post-quantum assumptions (lattice-based)
- Digital signatures: MUST be post-quantum (hash-based: SPHINCS+)
- Key encapsulation: MUST be post-quantum (lattice-based: ML-KEM/Kyber)
- Zero-knowledge proofs: MUST be post-quantum (hash-based: STARKs)
- Hash functions: MUST have quantum security (SHA-3/SHAKE256)

The following are PROHIBITED:

- Elliptic curve cryptography (ECDSA, EdDSA, ECDH)
- RSA
- Discrete logarithm-based systems
- Pairing-based cryptography
- Any system vulnerable to Shor's or Grover's algorithm beyond security margin

R2.2 No Trusted Setup [MANDATORY]

The system MUST NOT require any trusted setup ceremony.

There MUST NOT exist any "toxic waste" or trapdoor information
that could compromise the system if revealed.

All parameters MUST be publicly verifiable and deterministically derived.

R2.3 Cryptographic Binding [MANDATORY]

The commitment scheme MUST be computationally binding.

It MUST be computationally infeasible to open a commitment to two different values.

R2.4 Cryptographic Hiding [MANDATORY]

The commitment scheme MUST be computationally hiding.

A commitment MUST reveal no information about the committed value.

R2.5 Proof Soundness [MANDATORY]

The zero-knowledge proof system MUST have soundness error $< 2^{-100}$.

It MUST be computationally infeasible to generate a valid proof
for a false statement.

R2.6 Proof Zero-Knowledge [MANDATORY]

The zero-knowledge proof MUST reveal nothing beyond the truth of the statement.

There MUST exist a simulator that can produce indistinguishable proofs without knowledge of the witness.

R3. DECENTRALIZATION REQUIREMENTS

R3.1 Permissionless Participation [MANDATORY]

Anyone MUST be able to:

- Run a full node
- Validate the blockchain
- Create transactions
- Participate in consensus (mining)

There MUST NOT be any registration, approval, or permission required.

R3.2 No Privileged Parties [MANDATORY]

There MUST NOT exist any party with special privileges including:

- Ability to censor transactions
- Ability to reverse transactions
- Ability to mint coins outside of consensus rules
- Ability to modify protocol rules unilaterally
- Access to backdoors or master keys

R3.3 Permissionless Mining [MANDATORY]

Mining MUST be permissionless (no approval required to participate).
The mining algorithm MUST support high hashrate for network security.
No single entity SHALL control majority of mining infrastructure.
Mining algorithm choice: See Design Philosophy and Appendix H.3.

R3.4 Open Source [MANDATORY]

All protocol specifications MUST be public.
All reference implementations MUST be open source.
There MUST NOT be any proprietary components required for participation.

R4. INTEGRITY REQUIREMENTS

R4.1 Fixed Supply [MANDATORY]

Maximum supply: 21,000,000 QTM
This limit MUST NOT be changed.

This limit **MUST** be enforced by consensus rules.
There **MUST NOT** exist any mechanism to create coins beyond this limit.

R4.2 No Inflation Bugs [MANDATORY]

The system **MUST** mathematically guarantee that:

- No transaction can create value from nothing
- Sum of inputs = Sum of outputs + fee (always)
- This property **MUST** be enforced by zero-knowledge proofs

R4.3 Double-Spend Prevention [MANDATORY]

Each output **MUST** be spendable exactly once.

The nullifier mechanism **MUST** deterministically prevent double-spending.
This **MUST** be enforced at consensus level.

R4.4 Transaction Finality [MANDATORY]

Once a transaction is confirmed with sufficient depth,
it **MUST** be computationally infeasible to reverse.
DAG ordering **MUST** follow GhostDAG rules (highest blue score wins).

R5. FUNCTIONAL REQUIREMENTS

R5.1 Basic Transaction Support [MANDATORY]

The system **MUST** support:

- Multiple inputs per transaction (16)
- Multiple outputs per transaction (16)
- Variable transaction fees
- Memo fields for recipient

R5.2 Wallet Functionality [MANDATORY]

The system **MUST** support:

- Deterministic key derivation from seed phrase
- Balance scanning using view keys only
- Transaction creation using spend keys
- View key sharing for audit purposes (without spend capability)

R5.3 Light Client Support [MANDATORY]

The system **MUST** support light clients that can:

- Verify transaction inclusion via Merkle proofs

- Scan for owned outputs without full chain
 - Operate with privacy guarantees intact
-

R6. PERFORMANCE REQUIREMENTS

R6.1 Transaction Processing [MANDATORY]

Proof generation: MUST complete in < 60 seconds for a typical (2-in/2-out) transaction and < 120 seconds for a maximum-size (16-in/16-out) transaction on reference hardware

Proof verification: MUST complete in < 1 second per individual proof

Batch verification: amortized cost MUST be < 10 ms per transaction via aggregated proofs (Section 11.4) - REQUIRED for target throughput; 1,000 TPS × 1 s/proof cannot be sustained on commodity hardware

Block validation: MUST complete in < 10 seconds for 1000 transactions (using aggregated/batch verification)

R6.2 Storage [MANDATORY]

The system MUST be operable on hardware with:

- 4 TB NVMe storage for a pruned full node
- 32 GB RAM
- 8-core CPU
- 1 Gbps network connectivity (sustained target throughput, see G.1)

Notes: - “Full node” here refers to a pruned full node that retains the current state and a bounded recent history. Archival nodes require petabyte-scale storage at target throughput (see G.1). - The nullifier set can never be pruned (spent-output detection requires the full set). At 1,000 TPS with 2 inputs/tx this grows ~2 TB/year and dominates long-run pruned-node storage. Accumulator-based representations are a research direction (H.4).

R6.3 Network [MANDATORY]

Block rate: 10-32 blocks per second (DAG consensus)

Transaction throughput: 1,000 TPS sustained (see R8.2)

Confirmation time: < 10 seconds for high-confidence finality

Block propagation: < 1 second to 90% of nodes

R7. NON-REQUIREMENTS (Explicitly Excluded)

R7.1 NOT Required

The following are explicitly NOT requirements:

- Smart contracts (out of scope for v1)
- Governance tokens (no on-chain governance)
- Staking mechanisms (PoW only for v1)
- Regulatory compliance features
- Selective disclosure (privacy is absolute)
- Identity systems
- Interoperability with other chains (future work)

R7.2 NOT Permitted

The following MUST NOT be implemented:

- Backdoors for any party including developers or governments
 - Transaction censorship mechanisms
 - Blacklisting of addresses or outputs
 - "View-only" regulatory access without key holder consent
 - Inflationary monetary policy
 - Centralized components (oracles, coordinators, sequencers)
-

R8. SCALABILITY REQUIREMENTS

R8.1 DAG-Native Architecture [MANDATORY]

The protocol MUST use DAG-based consensus (GhostDAG or equivalent).
The protocol MUST NOT use sequential single-chain block ordering.
Parallel block creation MUST be supported natively.
All valid blocks MUST contribute to consensus (no orphans discarded).

Research Challenge: Privacy-preserving proofs over DAG structures.
This is a core research goal, not a deferrable optimization.

R8.2 L1 Throughput Targets [MANDATORY]

Block rate: 10 blocks per second (target: 32 blocks/second)
Transaction throughput: 1,000 TPS sustained on L1
Peak capacity: 5,000 TPS burst without failure
Confirmation time: < 10 seconds for high-confidence finality

Comparative baseline: Match or exceed Kaspas throughput
while adding privacy and quantum security.

R8.3 Parallel Processing [MANDATORY]

Transaction validation MUST support parallel execution.
STARK proof verification MUST be parallelizable across blocks.

Nullifier checking MUST scale with DAG width.
Merkle tree updates MUST handle concurrent block insertions.

R8.4 Privacy-Preserving DAG Consensus [MANDATORY]

The DAG structure MUST NOT weaken privacy guarantees:

- Anonymity set: MUST remain the full output set across all DAG branches
- Transaction graph: DAG parent references MUST NOT enable taint analysis
- Nullifier ordering: MUST prevent double-spend across parallel blocks
- Proof validity: STARKs MUST prove membership across DAG structure

This is the core research contribution of Quantum:

Solving privacy-preserving consensus over parallel block structures.

R8.5 Horizontal Scaling [MANDATORY]

Node operations MUST scale horizontally:

- Validation load MUST be distributable across CPU cores
- Block propagation MUST use DAG-aware protocols
- State synchronization MUST handle DAG branch merging
- Mempool MUST support high-throughput transaction ingestion (10K+ TPS)

R8.6 Layer 2 Compatibility [MANDATORY]

The protocol MUST support L2 scaling in addition to L1:

- Payment channels: Off-chain transactions with on-chain settlement
- Validity rollups: Batched proofs for higher throughput
- State channels: Generalized off-chain state transitions

L2 solutions provide: 100,000+ TPS for specific use cases.

L1 provides: Base layer security, finality, and censorship resistance.

Requirement Compliance Matrix

Requirement	Category	Verification Method
R1.1	Privacy	Code review: no transparent tx mode exists
R1.2	Privacy	Formal proof: anonymity set = all outputs
R1.3	Privacy	Formal proof: output-address unlinkability
R1.4	Privacy	Formal proof: commitment hiding property
R1.5	Privacy	Code review: Dandelion++ implementation
R2.1	Security	Audit: all primitives post-quantum
R2.2	Security	Code review: no trusted setup

Requirement	Category	Verification Method
R2.3	Security	Formal proof: commitment binding
R2.4	Security	Formal proof: commitment hiding
R2.5	Security	Formal proof: STARK soundness
R2.6	Security	Formal proof: STARK zero-knowledge
R3.1	Decentralization	Functional test: open participation
R3.2	Decentralization	Code review: no privileged keys
R3.3	Decentralization	Analysis: Permissionless mining, hashrate distribution
R3.4	Decentralization	License review: open source
R4.1	Integrity	Code review: supply cap in consensus
R4.2	Integrity	Formal proof: balance preservation
R4.3	Integrity	Formal proof: nullifier uniqueness
R4.4	Integrity	Analysis: finality properties
R5.x	Functional	Integration tests
R6.x	Performance	Benchmarks on reference hardware
R8.1	Scalability	Architecture review: DAG consensus implemented
R8.2	Scalability	Load testing: 1000+ TPS sustained
R8.3	Scalability	Benchmark: parallel validation scaling
R8.4	Scalability	Formal proof: privacy preserved over DAG
R8.5	Scalability	Integration test: horizontal node scaling
R8.6	Scalability	Design review: L2 compatibility

Immutability Declaration

These requirements constitute the immutable core of the Quantum specification.

SHA-256 hash of requirements section (R1-R8):

Computed over the IMMUTABLE REQUIREMENTS section of this document.

Draft v1.0 hash: [to be computed when specification is finalized]

Note: Hash will be updated when specification is finalized.

Any modification to R1-R8 MUST update this hash.

Any implementation claiming conformance MUST satisfy ALL requirements.

Partial conformance is not recognized.

"Almost quantum-secure" is not quantum-secure.

"Mostly private" is not private.

These requirements are binary: satisfied or not satisfied.

There is no middle ground.

END OF IMMUTABLE REQUIREMENTS

Part I: Cryptographic Foundation

Part I defines the cryptographic building blocks that underpin Quantum. Each component is selected for its quantum resistance and well-understood security properties. Together, these primitives enable private transactions with amounts hidden, senders unlinkable, and receivers unidentifiable—all without trusted setup or quantum-vulnerable assumptions.

The components build on each other: hash functions provide the foundation for domain-separated operations; polynomial rings enable efficient lattice arithmetic; commitments hide values while preserving verifiability; signatures authorize spending; key encapsulation enables secure one-time addressing; and STARKs prove transaction validity without revealing private data.

1. Notation and Conventions

1.1 Mathematical Notation

	Integers
$\mathbb{Z}_q$	Integers modulo q
$\mathbb{Z}_q[X]$	Polynomial ring over $\mathbb{Z}_q$
R_q	$\mathbb{Z}_q[X]/(X^n + 1)$ for $n = \text{power of } 2$
$[a, b]$	Closed interval from a to b
$\{0,1\}^n$	Bit strings of length n
$\{0,1\}^*$	Bit strings of arbitrary length
$ $	Concatenation
$ x $	Bit length of x
	XOR operation
$\leftarrow \$$	Sample uniformly at random
$_c$	Computationally indistinguishable

1.2 Security Parameter

$\kappa = 256$	Primary security parameter
	Targets 128-bit post-quantum security overall
	(hashes at 128-bit PQ; KEM/signatures use NIST Level 5 parameters)

1.3 Endianness and Encoding

All integers: Little-endian byte encoding

Field elements: Little-endian coefficient encoding

Points/Vectors: Concatenated element encodings

Structures: Deterministic serialization (see Section 12)

2. Hash Functions

Hash functions are the most fundamental cryptographic primitive in Quantum. Unlike elliptic curve operations, hash functions remain secure against quantum computers—Grover’s algorithm provides only a quadratic speedup, which is addressed by using 256-bit security parameters that yield 128-bit post-quantum security.

Why SHAKE256? We use SHAKE256 (SHA-3 family) as the universal hash function because: - **Extendable output:** Can produce arbitrary-length output, simplifying API design - **Domain separation:** Different hash instances are created by prefixing distinct domain tags - **No length extension attacks:** Unlike SHA-2, SHA-3’s sponge construction prevents length extension - **NIST standardized:** FIPS 202 provides implementation confidence

2.1 Primary Hash Function: SHAKE256

Definition: SHAKE256 is the extendable-output function from SHA-3 (FIPS 202).

$$H: \{0,1\}^* \times \mathbb{N} \rightarrow \{0,1\}^*$$

$$H(m, \ell) = \text{SHAKE256}(m, \ell)$$

Where ℓ is the output length in bits.

Domain Separation: All hash function calls use domain-separated inputs:

$$H_{\text{domain}}(x) = H(\text{encode}(\text{"Quantum-v1."} \parallel \text{domain}) \parallel x, \text{output_len})$$

Where $\text{encode}(s) = \text{len}(s)$ as 2-byte LE $\parallel s$ as UTF-8 bytes

2.2 Defined Hash Instances

Instance	Domain Tag	Output Length	Usage
H_commitment	"commitment"	512 bits	Commitment randomness
H_nullifier	"nullifier"	256 bits	Nullifier derivation
H_merkle	"merkle"	256 bits	Merkle tree hashing
H_address	"address"	256 bits	Address derivation
H_kdf	"kdf"	variable	Key derivation
H_challenge	"challenge"	512 bits	Fiat-Shamir challenges
H_pow	"pow"	256 bits	Proof of work

2.3 Hash-to-Field

HashToField(m, q, k):

Input: message m , modulus q , count k

Output: k elements in $\mathbb{F}_q$

1. $\text{bits} = \log_2(q) + 128$ // Extra bits for uniform reduction
 2. For i in $0..k$:
 - $\text{bytes}_i = H(\text{encode}(\text{"h2f"}) \parallel m \parallel i \text{ as 1-byte, })$
 - $z_i = \text{bytes_to_integer}(\text{bytes}_i) \bmod q$
 3. Return $(z_0, \dots, z_{k-1})$
-

3. Lattice-Based Commitments

Commitments are how Quantum hides transaction amounts while proving they balance. A commitment scheme allows you to “commit” to a value (like a transaction amount) in a way that hides the value but binds you to it—you cannot later claim you committed to a different value.

Why lattice-based? Traditional Pedersen commitments use elliptic curve points, which are broken by quantum computers. Lattice-based commitments achieve the same functionality under quantum-hard assumptions. The security reduces to the Module Learning With Errors (Module-LWE) problem: given noisy linear equations over polynomial rings, recover the secret. This problem resists all known classical and quantum algorithms.

The key property: Commitments are *additively homomorphic*. If $C(a)$ commits to value a and $C(b)$ commits to value b , then $C(a) + C(b) = C(a+b)$. This allows us to verify that inputs equal outputs plus fee without decrypting any amounts.

3.1 Module-LWE Parameters

Ring Definition:

```
n = 256                                // Polynomial degree
q = 8380417                             // Prime modulus ( 2^23)
R_q = _q[X]/(X^n + 1)                   // Polynomial ring

k = 4                                    // Module rank for commitments
= 2                                      // Secret/noise coefficient bound
```

Rationale: These parameters align with CRYSTALS-Dilithium-style module parameters (q, n, k) . The target is ≥ 128 -bit post-quantum security, subject to formal analysis.

3.2 Polynomial Operations

Addition in R_q :

$$(a + b)_i = (a_i + b_i) \bmod q$$

Multiplication in R_q (NTT-based):

$$a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \cdot \text{NTT}(b))$$

Where $\cdot$ is coefficient-wise multiplication

NTT: Number Theoretic Transform

Using primitive 512th root of unity $\omega = 1753$ in R_q

3.3 Commitment Scheme

Key Generation (public parameters):

Setup(1^λ):

1. $A \leftarrow R_q^{k \times k}$ // Random matrix (can be derived from seed)
2. Return $pp = A$

Commit:

Commit(pp, v, r):

Input:

- $pp = A$ (public parameters)
- $v \in [0, 2^{64})$ (transaction amount in base units)
- $r \in R_q^k$ (randomness vector with small coefficients)

Value encoding (REQUIRED - a single coefficient cannot hold a 64-bit amount, since $q = 2^{23}$):

Decompose v into 4 limbs of 16 bits: $v = \sum_{j=0}^3 v_j \cdot 2^{16j}$

$\text{Enc}(v) = \text{polynomial with coefficients } (v_0, v_1, v_2, v_3, 0, \dots, 0)$

Constraint: All coefficients of r_i must be in $[-1, 1]$

Output:

$$c = A \cdot r + \text{Enc}(v) \cdot e_1 \in R_q^k$$

Where $e_1 = (1, 0, \dots, 0)^T$

Return (c, r) // c is commitment, r is opening

Note on limb headroom: 16-bit limbs in a 23-bit modulus leave 2^7 of headroom, so limb-wise sums of up to 128 commitments stay below q without wrap-around. Consensus-level balance is NOT derived from modular commitment sums - it is proven over the integers inside the STARK (Section 10.1, constraints 1, 2, 6), so no value can be created mod q .

Verify Opening:

VerifyOpening(pp, c, v, r, ℓ):

Input: ℓ = number of homomorphically accumulated commitments ($\ell = 1$)

```

    for a fresh commitment)
1. Check all coefficients of  $r_i$  are in  $[-\cdot, \cdot]$ 
2. Check  $c == A \cdot r + \text{Enc}(v) \cdot e_1$ 
3. Return accept/reject

```

Note (relaxed openings): adding commitments adds their randomness, so the opening bound grows linearly with the number of summands. Binding must therefore be analyzed for relaxed openings with bound $B = \cdot$, as in BDLOP-style lattice commitments. The formal analysis (Phase 1) MUST establish the maximum for which Module-SIS binding still holds.

Properties: - **Hiding:** Computationally hiding under Module-LWE - **Binding:** Computationally binding under Module-SIS (for relaxed openings up to bound $\ell \cdot \eta$, see above) - **Homomorphic:** $\text{Commit}(v1, r1) + \text{Commit}(v2, r2) = \text{Commit}(v1+v2, r1+r2)$, valid while limb sums stay below q (≤ 128 summands) and the opening bound holds

3.4 Randomness Generation

```

GenerateCommitmentRandomness(seed):
1. expanded = H_commitment(seed)
2. For i in 0..k:
    For j in 0..n:
        // Sample coefficient in  $[-, ]$ 
        byte = expanded[i*n + j]
        coeff = (byte mod (2+1)) -
        r[i][j] = coeff
3. Return r

```

4. Hash-Based Signatures: SPHINCS+-256f

4.1 Parameters

Using SPHINCS+-SHAKE-256f-simple (NIST standardized):

```

n = 32           // Hash output length (bytes)
h = 68           // Total tree height
d = 17           // Hypertree layers
a = 9            // FORS tree height
k = 35           // FORS trees
w = 16           // Winternitz parameter

```

Signature size: 49,856 bytes

Public key size: 64 bytes

Secret key size: 128 bytes

4.2 API

SPHINCS_KeyGen(seed):

Input: 96-byte seed

Output: (pk, sk)

// As specified in SPHINCS+ documentation

SPHINCS_Sign(sk, m):

Input: secret key sk, message m

Output: signature (49,856 bytes)

SPHINCS_Verify(pk, m,)::

Input: public key pk, message m, signature

Output: accept/reject

4.3 Security

- Post-quantum secure under hash function security assumptions
 - No algebraic structure to attack
 - Stateless (unlike XMSS)
-

5. Key Encapsulation: ML-KEM-1024 (Kyber)

5.1 Parameters

Using ML-KEM-1024 (NIST FIPS 203):

n = 256 // Polynomial degree

k = 4 // Module rank

q = 3329 // Modulus

1 = 2 // Secret key noise

2 = 2 // Ciphertext noise

Public key: 1,568 bytes

Secret key: 3,168 bytes

Ciphertext: 1,568 bytes

Shared secret: 32 bytes

5.2 API

Kyber_KeyGen():

Output: (pk, sk)

Kyber_Encapsulate(pk):

Output: (ciphertext, shared_secret)

```
Kyber_Decapsulate(sk, ciphertext):
    Output: shared_secret
```

6. Zero-Knowledge Proofs: STARKs

Zero-knowledge proofs are the cryptographic core of Quantum’s privacy guarantees. They allow a prover to convince a verifier that a statement is true (e.g., “this transaction is valid and balanced”) without revealing *anything* beyond that fact—not the amounts, not which outputs were spent, not the sender or receiver.

Why STARKs specifically? The choice of STARK over other ZK systems (SNARKs, Bulletproofs) is driven by two requirements:

1. **No trusted setup:** SNARKs require a ceremony where participants generate parameters and destroy secrets. If anyone cheats, they can forge proofs forever. STARKs derive all parameters from public randomness, eliminating this catastrophic risk.
2. **Quantum security:** SNARKs typically rely on elliptic curve pairings or discrete log assumptions, both broken by quantum computers. STARKs rely only on collision-resistant hash functions, which remain secure.

The trade-off is proof size: STARK proofs are larger (~100KB vs ~1KB). For a system designed to remain secure for decades, this is acceptable.

6.1 Overview

STARKs (Scalable Transparent Arguments of Knowledge) provide: - **Transparency:** No trusted setup
- **Post-quantum security:** Based only on hash functions - **Scalability:** Polylogarithmic verification

6.2 Arithmetic Intermediate Representation (AIR)

Computations are expressed as:

AIR Definition:

- Trace width: w (number of columns)
- Trace length: $T = 2^t$ (power of 2)
- Transition constraints: Polynomial relations between consecutive rows
- Boundary constraints: Values at specific positions

6.3 Field Selection

Prime field: $p = 2^{64} - 2^{32} + 1$ (Goldilocks prime)

Properties:

- Efficient 64-bit arithmetic

- 2^{32} roots of unity (enables large FFTs)
- Suitable for recursive STARKs

6.4 FRI Parameters (Fast Reed-Solomon IOP)

Blowup factor: = 8

Number of queries: 34

Grinding bits: 20

Folding factor: 4

Resulting security: ~122 bits conjectured

(34 queries $\times$ $\log_2(8)$ bits/query + 20 grinding bits = 122,
exceeding the 2^{-100} soundness target with margin)

6.5 STARK Proof Structure

```
struct StarkProof {
    // Commitments
    trace_commitment: [u8; 32],
    constraint_commitment: [u8; 32],
    fri_commitments: Vec<[u8; 32]>,

    // Query responses
    trace_queries: Vec<TraceQuery>,
    fri_queries: Vec<FriQuery>,

    // Final layer
    fri_final: Vec<FieldElement>,

    // Proof of work (grinding)
    pow_nonce: u64,
}
```

Approximate size: 50-200 KB depending on statement complexity

7. Merkle Trees (Quantum-Secure)

7.1 Construction

Binary Merkle tree using H_{merkle} :

```
MerkleHash(left, right):
    Return  $H_{\text{merkle}}(0x00 \parallel \text{left} \parallel \text{right})$ 
```

```
LeafHash(data):
    Return H_merkle(0x01 || data)
```

Domain prefixes: 0x00 = internal node, 0x01 = leaf,
0x02 = note commitment (Section 9.1)

7.2 Tree Parameters

Depth: 40 (supports 2^{40} 1 trillion leaves)

Node size: 32 bytes

Proof size: $40 \times 32 = 1,280$ bytes

7.3 Append-Only Tree

```
struct MerkleTree {
    depth: u32,
    leaves: Vec<[u8; 32]>,
    nodes: Vec<Vec<[u8; 32]>>, // nodes[level][index]
}

impl MerkleTree {
    fn append(&mut self, leaf: [u8; 32]) -> u64 {
        let index = self.leaves.len() as u64;
        self.leaves.push(LeafHash(leaf));
        self.recompute_path(index);
        index
    }

    fn root(&self) -> [u8; 32] {
        self.nodes[self.depth as usize][0]
    }

    fn prove(&self, index: u64) -> MerkleProof {
        // Return sibling hashes along path to root
    }
}
```

Part II: Protocol Specification

Part II specifies how the cryptographic primitives from Part I combine into a working blockchain protocol. This includes the account model, transaction lifecycle, consensus mechanism, and network layer.

How a Transaction Works (High-Level):

1. **Creating outputs:** The sender generates commitments to amounts, encrypts output data to recipients using their view keys, and creates new “notes” (outputs)
2. **Spending inputs:** To spend previous outputs, the sender reveals nullifiers (deterministic tags that mark outputs as spent) without revealing *which* outputs they correspond to
3. **Proving validity:** The sender generates a STARK proof that the transaction is balanced (inputs = outputs + fee), all spent outputs existed, and all nullifiers are correctly formed
4. **Authorization:** The sender signs the transaction sighash with the spend keys bound into the spent notes; the signatures are verified *inside* the STARK proof, so the keys are never revealed on-chain (revealing them would link spends by the same wallet)
5. **Propagation:** The transaction propagates through the network using Dandelion++ to hide the sender’s IP address
6. **Inclusion:** Miners validate the proof, check nullifiers aren’t already spent, and include the transaction in a block

This design provides complete privacy: observers see only nullifiers (unlinkable to outputs) and new commitments (hiding amounts and recipients).

8. Account and Address System

8.1 Key Hierarchy

MasterSeed: 256 bits (from CSPRNG or BIP39)

→ $H_kdf("spend" || MasterSeed, 256) \rightarrow SpendSeed$

→ $SPHINCS_KeyGen(H_kdf("spx" || SpendSeed, 768)) \rightarrow (SpendPK, SpendSK)$
 → $SpendPKHash = H_address("spend-pk" || SpendPK)$

→ $H_kdf("nullifier-key" || SpendSeed, 256) \rightarrow NullifierKey (nk, secret)$
 → $NullifierPK = H_address("nk-pub" || nk)$

→ $H_kdf("view" || MasterSeed, 256) \rightarrow ViewSeed$

→ $Kyber_KeyGen(H_kdf("kem" || ViewSeed, 512)) \rightarrow (ViewPK, ViewSK)$

Notes:

- Seeds are expanded via H_kdf to the exact seed lengths the primitives require: 768 bits (96 bytes) for SPHINCS+, 512 bits (64 bytes) for ML-KEM.
- nk MUST remain secret. $NullifierPK = H(nk)$ is published in the address and is bound into every note created for this wallet (Section 9.1). This is what prevents anyone other than the recipient - including the original sender, who knows the value commitment's opening - from deriving a valid nullifier.

8.2 Address Format

Address = (SpendPK, ViewPK, NullifierPK)

Serialized:

SpendPK: 64 bytes (SPHINCS+ public key)
 ViewPK: 1,568 bytes (ML-KEM-1024 public key)
 NullifierPK: 32 bytes (hash of nullifier key, see Section 8.1)
 Total: 1,664 bytes

Encoded: "qtm" || version byte || Base32(payload || checksum)
 checksum = first 8 bytes of H("addr-checksum" || version || payload)

Note: Bech32m is deliberately NOT used. Its checksum guarantees (BIP-350) only hold up to 90 characters; a 1,664-byte payload far exceeds that limit.

Shortened address (for display):

First 32 bytes of H("address-short" || Address)
 Used for human verification, not transactions

8.3 Stealth Addresses

For each transaction output, sender generates one-time address:

GenerateStealthAddress(RecipientViewPK):

1. (Ciphertext, SharedSecret) = Kyber_Encapsulate(RecipientViewPK)
2. OneTimeKey = H_address(SharedSecret)
3. Return (Ciphertext, OneTimeKey)

Recipient scanning:

ScanOutput(ViewSK, Ciphertext, EncryptedData):

1. SharedSecret = Kyber_Decapsulate(ViewSK, Ciphertext)
2. OneTimeKey = H_address(SharedSecret)
3. DecryptionKey = H_kdf("decrypt" || OneTimeKey, 256)
4. Parse EncryptedData = nonce || ciphertext || tag
5. Data = AES256_GCM_Decrypt(DecryptionKey, nonce, ciphertext, tag)
6. If decryption succeeds, output belongs to us
7. Return Data or

9. Transaction Structure

9.1 Output (Note)

```
struct Output {
```

```

    // Public (stored on-chain)
    commitment: LatticeCommitment,      // k × n coefficients in _q (~3,072 bytes)
    note_commitment: [u8; 32],          // Ownership binding; the Merkle tree leaf
    kyber_ciphertext: [u8; 1568],        // For stealth address
    encrypted_data: [u8; 148],           // AES-GCM: nonce(12) || ciphertext(120) || tag(16)

    // Size: approximately 4.8 KB per output
}

// Note commitment (ownership binding):
// s = H_kdf("note-blind" || SharedSecret, 256) // known to sender and recipient
// note_commitment = H_merkle(0x02 || serialize(commitment)
//                               || SpendPKHash_recipient || NullifierPK_recipient || s)
//
// The note commitment - not the raw value commitment - is the leaf inserted
// into the output Merkle tree. It binds the output to the recipient's spend
// and nullifier keys without revealing them (the blinding s prevents
// dictionary tests against known addresses). Spending requires proving this
// binding in zero-knowledge (Section 10.1, constraints 7-9), so knowledge of
// the value commitment's opening alone - which the sender has - is NOT
// sufficient to spend. Recipients recompute note_commitment during scanning
// to confirm ownership.

// Encrypted data plaintext structure:
struct OutputPlaintext {
    value: u64,                // 8 bytes
    blinding_seed: [u8; 32],   // 32 bytes, expands to full randomness
    memo: [u8; 64],           // 64 bytes, arbitrary user data
    checksum: [u8; 16],        // 16 bytes, for integrity
}

// Encryption (deterministic nonce):
// nonce = H_kdf("nonce" || OneTimeKey, 96) // 12 bytes
// ciphertext, tag = AES256_GCM_Encrypt(DecryptionKey, nonce, OutputPlaintext)
// encrypted_data = nonce || ciphertext || tag

```

9.2 Nullifier

ComputeNullifier(NullifierKey, Commitment, Position):

Input:

NullifierKey: 256-bit key from wallet
 Commitment: The output's commitment (serialized)
 Position: u64 index in global output list

Output:

```
H_nullifier(NullifierKey || Commitment || Position.to_le_bytes())
```

Size: 32 bytes

9.3 Transaction

```
struct Transaction {
    // Inputs (spent outputs)
    nullifiers: Vec<[u8; 32]>,

    // Outputs (new notes)
    outputs: Vec<Output>,

    // Fee (public, in base units)
    fee: u64,

    // STARK proof of validity AND spend authorization
    validity_proof: StarkProof,

    // Merkle root at time of creation
    anchor: [u8; 32],
}

// There is NO on-chain signature structure. Spend authorization is part of
// the STARK statement (Section 10.1, constraint 8): the prover supplies the
// SPHINCS+ public keys and signatures as private witness, and the circuit
// verifies them against the sighash and the key hashes bound into the spent
// notes. Publishing the signatures or public keys would link all spends by
// the same wallet; verifying them in-circuit preserves unlinkability and
// removes ~50 KB per input from the on-chain transaction.
//
// sighash = H_merkle(serialization of nullifiers || outputs || fee || anchor)
// The sighash is a public input of the proof, which binds the proof to this
// exact transaction content (non-malleability).
```

9.4 Transaction Size Estimate

2-input, 2-output transaction:

- Nullifiers: $2 \times 32 = 64$ bytes
- Outputs: $2 \times 4,820 = 9,640$ bytes
- Fee: 8 bytes
- STARK proof: ~100,000 bytes
- Anchor: 32 bytes
- Overhead: ~100 bytes

Total: ~110 KB per transaction

(SPHINCS+ signatures are witness data verified inside the proof and do not appear on-chain; see Section 9.3.)

10. Validity Proof (STARK Circuit)

10.1 Statement to Prove

For a transaction with m inputs and n outputs:

Public inputs:

- nullifiers[0.. m]: Nullifiers of spent outputs
- output_commitments[0.. n]: Value commitments of new outputs
- output_note_commitments[0.. n]: Note commitments of new outputs
- fee: Transaction fee
- anchor: Merkle root
- sighash: Hash of the transaction content (Section 9.3)

Private inputs (witness):

- input_values[0.. m]: Values of spent outputs
- input_blindings[0.. m]: Blinding factors of spent outputs
- input_positions[0.. m]: Positions in Merkle tree
- input_merkle_paths[0.. m]: Merkle authentication paths
- input_note_blinds[0.. m]: Note blindings s (Section 9.1)
- input_nullifier_keys[0.. m]: Nullifier keys nk
- input_spend_pks[0.. m]: SPHINCS+ public keys of spent notes
- input_spend_sigs[0.. m]: SPHINCS+ signatures over sighash
- output_values[0.. n]: Values of new outputs
- output_blindings[0.. n]: Blinding factors of new outputs
- output_owner_data[0.. n]: (SpendPKHash, NullifierPK, s) per new note

Constraints:

1. Balance: $\sum \text{input_values} = \sum \text{output_values} + \text{fee}$
2. Range: $i: 0 \leq \text{output_values}[i] < 2^{64}$ (16-bit limb decomposition, Section 3.3)
3. Commitments: $j: \text{output_commitments}[j] = \text{Commit}(\text{output_values}[j], \text{output_blindings}[j])$ and $\text{output_note_commitments}[j]$ correctly binds $\text{output_owner_data}[j]$ (Section 9.1)
4. Nullifiers: $i: \text{nullifiers}[i] = H_{\text{nullifier}}(\text{input_nullifier_keys}[i] || \text{input_commitments}[i])$
5. Membership: $i: \text{MerkleVerify}(\text{input_note_commitments}[i], \text{input_positions}[i], \text{input_merkle_paths}[i])$ where $\text{input_note_commitments}[i]$ is recomputed in-circuit from the witness (constraint 3)
6. No overflow: $\sum \text{input_values} < 2^{64}$ (prevent wrap-around)
7. Note binding: $i: \text{input_note_commitments}[i] = H_{\text{merkle}}(0x02 || \text{input_commitments}[i] || H_{\text{address}}(\text{"spend-pk"} || \text{input_spend_pks}[i]) || H_{\text{address}}(\text{"nk-pub"} || \text{input_nullifier_keys}[i]) || \text{input_note_blinds}[i])$

- (ties the spent note to the spend key AND proves the witness `nk` is the one the note was created for - the sender of a note cannot satisfy this with their own `nk`, which closes the re-spend/double-nullifier attack)
8. Spend authorization: `i`: `SPHINCS_Verify(input_spend_pks[i], sighash, input_spend_s)` (verified in-circuit; SPHINCS+ verification is pure hashing and therefore STARK-friendly, but its trace cost is a primary engineering challenge - see H.1)
 9. Sighash binding: `sighash` is a public input; any change to nullifiers, outputs, fee, or anchor invalidates the proof (non-malleability)

10.2 AIR Constraints (Detailed)

```
// Trace layout (columns)
Column 0-3: Input value decomposition (4 × 16-bit limbs per value, Section 3.3)
Column 4-7: Output value decomposition
Column 8-79: Commitment verification
Column 80-119: Merkle path verification
Column 120-127: Hash computation state (note binding, nullifiers, SPHINCS+ verification)

// Transition constraints (polynomial degree 8)
// Balance constraint (accumulator pattern; fee subtracted in final row):
trace[i+1][ACC] = trace[i][ACC] + trace[i][INPUT_VAL] - trace[i][OUTPUT_VAL]

// Range constraint (16-bit limbs):
// A naive product constraint over all limb values would have degree 2^16,
// violating the degree-8 bound. Limbs are instead range-checked via a
// lookup argument against a precomputed 16-bit table (or, equivalently,
// constrained by full bit decomposition with degree-2 booleanity checks).

// Commitment constraint:
// Verify lattice multiplication step-by-step
// A · r computation spread across multiple rows

// Merkle constraint:
// Hash compression function computed row-by-row
// Path verification via conditional selection
```

10.3 Proof Generation

```
GenerateTransactionProof(public_inputs, witness):
1. Construct execution trace T (matrix of field elements)
2. Interpolate trace into polynomials
3. Compute constraint composition polynomial
4. Commit to trace and constraint polynomials
5. Run FRI protocol for low-degree testing
```

6. Apply Fiat-Shamir to make non-interactive
7. Add proof-of-work grinding
8. Return StarkProof

10.4 Proof Verification

```
VerifyTransactionProof(public_inputs, proof):
    1. Reconstruct Fiat-Shamir challenges
    2. Verify proof-of-work nonce
    3. Check trace commitment matches queries
    4. Verify constraint evaluations at query points
    5. Verify FRI layers
    6. Check FRI final layer is low-degree
    7. Verify boundary constraints from public inputs
    8. Return accept/reject
```

11. Consensus: Proof of Work

Why Proof of Work? In a privacy-focused system, proof of stake creates problematic dynamics: stake is visible on-chain (compromising privacy) or requires trusted infrastructure to verify (compromising decentralization). Proof of work provides permissionless participation without revealing participant identity or holdings.

Why high hashrate matters: Higher total network hashrate makes 51% attacks more expensive. Bitcoin and Kaspero demonstrate that professional mining (including ASICs) can coexist with permissionless participation and geographic distribution. Security comes from the cost to attack, not the hardware type.

11.1 Hash Function

The mining algorithm is an **open research question** (see Design Philosophy and Appendix H.3). The general structure:

```
PowHash(header):
    1. classical_hash = MINING_HASH(header)    // kHeavyHash, SHA-256d, or to be defined
    2. quantum_hash = H_pow(classical_hash)    // SHAKE256 for quantum security
    3. Return quantum_hash
```

Candidates: - **kHeavyHash:** Proven for DAG (Kaspero), high hashrate, GPU/ASIC compatible - **SHA-256d:** Highest hashrate security (Bitcoin), extremely fast verification - **Custom:** Optimized for privacy-DAG requirements if needed

Rationale: The mining hash provides proof-of-work security through high hashrate. The additional SHAKE256 hash ensures quantum security of the final output. Final algorithm choice requires empirical testing at target block rates.

11.2 Block Header (DAG)

```

struct BlockHeader {
    version: u32,                // Protocol version
    parent_hashes: Vec<[u8; 32]>, // Hashes of parent blocks (DAG structure)
    num_parents: u8,             // Number of parents (typically 1-64)
    blue_score: u64,             // GhostDAG blue score
    merkle_root: [u8; 32],       // Merkle root of transactions
    output_tree_root: [u8; 32],  // Root of output Merkle tree
    nullifier_set_root: [u8; 32], // Root of nullifier accumulator
    timestamp: u64,              // Unix timestamp (milliseconds for DAG)
    difficulty: [u8; 32],        // Target difficulty (256-bit)
    nonce: u64,                  // PoW nonce

    // Variable size due to parent_hashes
    // Minimum: 140 bytes (1 parent)
    // Typical: 300-500 bytes (5-10 parents)
}

impl BlockHeader {
    fn hash(&self) -> [u8; 32] {
        H_merkle(self.serialize())
    }

    fn pow_valid(&self) -> bool {
        let pow_hash = PowHash(self.serialize());
        pow_hash < self.difficulty
    }
}

```

11.3 Difficulty Adjustment (DAG)

DAG-aware difficulty adjustment based on block rate:

```

AdjustDifficulty(dag_state):
    EPOCH_DURATION = 100    // Epoch duration in seconds
    TARGET_RATE = 10        // Target blocks per second
    EXPECTED_BLOCKS = EPOCH_DURATION * TARGET_RATE // 1000 blocks

    // Count blocks in epoch window
    epoch_start = current_time - EPOCH_DURATION
    blocks_in_epoch = count_blocks_since(epoch_start)

    // Calculate adjustment ratio
    ratio = blocks_in_epoch / EXPECTED_BLOCKS

```

```
// Smooth adjustment (DAG requires more stability)
adjustment = clamp(ratio, 0.75, 1.25) // Max 25% change per epoch

// header.difficulty is a TARGET (pow_valid: pow_hash < target).
// Too many blocks (ratio > 1) must make mining HARDER, i.e. SHRINK
// the target - divide, do not multiply.
new_target = previous_target / adjustment
Return new_target
```

Note: DAG requires smoother difficulty adjustments than sequential chains because high block rates amplify oscillations.

11.4 Block Structure

```
struct Block {
    header: BlockHeader,
    transactions: Vec<Transaction>,

    // Aggregated proof. REQUIRED for operation at target throughput:
    // per-transaction verification cannot sustain 1,000 TPS on commodity
    // hardware (see R6.1). Optional only at low transaction volume.
    aggregated_proof: Option<AggregatedStarkProof>,
}
```

11.5 Block Validation (DAG)

ValidateBlock(block, dag_state):

1. Check all header.parent_hashes exist in DAG
2. Check header.num_parents matches parent_hashes.len()
3. Check header.pow_valid()
4. Check header.timestamp > max(parent timestamps)
5. Check header.timestamp < current_time + 2 hours
6. Check header.blue_score is valid per GhostDAG rules
7. For each transaction tx in block.transactions:
 - a. Check tx.anchor references valid DAG block
 - b. Check all nullifiers are not in nullifier set
 - c. Verify tx.validity_proof (or the block's aggregated proof; spend authorization is part of the proof statement, Section 10.1)
8. Check header.merkle_root == MerkleRoot(block.transactions)
9. Check header.output_tree_root == updated output tree root
10. Check header.nullifier_set_root == updated nullifier set root
11. Return accept/reject

12. Serialization

12.1 Canonical Encoding

All structures use deterministic, canonical encoding:

Integers: Little-endian, fixed width

- u8: 1 byte
- u32: 4 bytes
- u64: 8 bytes
- u256: 32 bytes

Variable-length data:

- Length prefix: 4 bytes (u32, little-endian)
- Followed by: raw bytes

Arrays:

- Count prefix: 4 bytes (u32)
- Followed by: concatenated element encodings

Polynomials in R_q :

- n coefficients, each as 3 bytes (for $q < 2^{24}$)
- Total: 768 bytes per polynomial

Vectors in R_q^k :

- k polynomials concatenated
- Total: 3,072 bytes for k=4

12.2 Transaction Serialization

SerializeTransaction(tx):

```
result = []
result.append(u32_le(tx.nullifiers.len()))
for nullifier in tx.nullifiers:
    result.append(nullifier) // 32 bytes each

result.append(u32_le(tx.outputs.len()))
for output in tx.outputs:
    result.append(SerializeOutput(output))

result.append(u64_le(tx.fee))
result.append(SerializeStarkProof(tx.validity_proof))
result.append(tx.anchor) // 32 bytes
```

```
Return concat(result)
```

13. Network Protocol

13.1 Transport Layer

Protocol: Noise XX pattern instantiated with ML-KEM-1024
(Noise_XX_MLKEM1024_ChaChaPoly_SHA3, post-quantum KEM-based handshake)

Classical-only handshakes (X25519 or any elliptic-curve DH) are PROHIBITED - R2.1 applies to the transport layer as well. Recorded P2P traffic is subject to the same harvest-now-decrypt-later threat as on-chain data.

Port: 19333 (mainnet), 19334 (testnet)

Message framing:

Length: 4 bytes (u32, max 64 MB - must accommodate full blocks,
which reach tens of MB at target throughput; see G.1)
Type: 1 byte
Payload: Length - 1 bytes

13.2 Message Types

```
enum MessageType {  
    // Handshake  
    Version = 0x00,  
    VersionAck = 0x01,  
  
    // Peer discovery  
    GetPeers = 0x10,  
    Peers = 0x11,  
  
    // Block propagation  
    Inventory = 0x20,  
    GetBlocks = 0x21,  
    Block = 0x22,  
    GetHeaders = 0x23,  
    Headers = 0x24,  
  
    // Transaction propagation  
    Transaction = 0x30,  
    GetTransaction = 0x31,
```

```
// Dandelion++
DandelionTx = 0x40,
}
```

13.3 Dandelion++ Parameters

Stem probability: 0.9 (90% continue stem, 10% fluff)
Stem timeout: 60 seconds
Embargo timeout: 30 seconds
Stem peers: 2 outbound connections designated as stem

14. State Management

14.1 Chain State

```
struct DAGState {
    // Virtual selected parent chain (VSPC) tip
    virtual_selected_parent: [u8; 32],

    // All current DAG tips (blocks with no children)
    tips: HashSet<[u8; 32]>,

    // Blue score of highest-scoring block
    max_blue_score: u64,

    // Cumulative PoW weight
    cumulative_difficulty: U256,

    // Output tree (append-only Merkle tree)
    output_tree: MerkleTree,
    output_count: u64,

    // Nullifier set (for double-spend prevention)
    nullifier_set: HashSet<[u8; 32]>,

    // Recent block headers (for anchor validation)
    recent_headers: VecDeque<BlockHeader>, // Last 100 in VSPC
}
```

14.2 Database Schema

Key-Value Store (RocksDB or similar):

Blocks:

```
Key: "block:" || block_hash
Value: Serialized Block
```

Block index:

```
Key: "height:" || height.to_be_bytes()
Value: block_hash
```

Outputs:

```
Key: "output:" || position.to_be_bytes()
Value: Serialized Output
```

Output Merkle nodes:

```
Key: "merkle:" || level.to_u8() || index.to_be_bytes()
Value: 32-byte hash
```

Nullifiers:

```
Key: "nullifier:" || nullifier
Value: (empty, presence is sufficient)
```

DAG state:

```
Key: "state:dag"
Value: Serialized DAGState
```

15. Wallet Operations

15.1 Key Generation

GenerateWallet():

1. entropy = CSPRNG(256 bits)
2. mnemonic = BIP39_Encode(entropy) // 24 words
3. master_seed = PBKDF2-HMAC-SHA512(mnemonic, "Quantum", 100000, 256)
4. Derive keys per Section 8.1
5. Return Wallet { master_seed, keys }

15.2 Scanning for Outputs

ScanBlock(wallet, block):

```
for tx in block.transactions:
    for (i, output) in tx.outputs.enumerate():
        result = TryScanOutput(wallet.view_sk, output)
        if result != :
            (value, blinding_seed, memo) = result
            position = global_output_position(block, tx, i)
            wallet.add_output(output, value, blinding_seed, position)
```

15.3 Creating Transactions

```
CreateTransaction(wallet, recipients, fee):
    // Select inputs
    inputs = wallet.select_inputs(sum(recipients.values) + fee)

    // Create outputs for recipients
    outputs = []
    for (address, value) in recipients:
        output = CreateOutput(address, value)
        outputs.append(output)

    // Create change output if needed
    change = sum(inputs.values) - sum(recipients.values) - fee
    if change > 0:
        change_output = CreateOutput(wallet.address, change)
        outputs.append(change_output)

    // Compute sighash over the final transaction content (Section 9.3)
    sighash = H_merkle(Serialize(nullifiers || outputs || fee || anchor))

    // Sign sighash with the spend key of each spent note; signatures and
    // public keys go into the WITNESS, never on-chain (Section 10.1)
    spend_sigs = [SPHINCS_Sign(wallet.spend_sk, sighash) for each input]

    // Generate validity proof (includes in-circuit signature verification)
    witness = PrepareWitness(wallet, inputs, outputs, fee, spend_sigs)
    proof = GenerateTransactionProof(public_inputs(sighash, ...), witness)

    // Assemble transaction
    Return Transaction { nullifiers, outputs, fee, proof, anchor }
```

16. Economic Parameters

16.1 Supply Schedule

Distribution: Fair launch (no premine, no ICO, no founder's reward)
 Total supply: 21,000,000 QTM (cap; actual emission sums slightly below,
 as in Bitcoin, due to integer rounding)

The schedule is defined for the DAG block rate, NOT copied from Bitcoin's
 10-minute blocks. At the target rate of 10 blocks/second:

Initial block reward: 1,000,000 satoshi (0.01 QTM)

Halving interval: 1,050,000,000 blocks (~3.3 years at 10 blocks/second)

BlockReward(height):

```

    HALVING_INTERVAL = 1_050_000_000
    halvings = height / HALVING_INTERVAL
    if halvings >= 64:
        return 0
    return 1_000_000 >> halvings // satoshi; integer shift, rounds down

```

Sanity check:

```

    Σ emission = 1,050,000,000 × Σ_{i=0}^{19} floor(1,000,000 / 2^i)
                = 1,050,000,000 × 1,999,993 sat
                = 20,999,926 QTM (< 21,000,000 cap )
    Reward reaches 0 after 20 halvings (~66 years of emission).

```

Block-rate invariance: emission is defined per unit time (10^7 sat/second initially). If a consensus parameter change alters the target block rate R , the per-block reward MUST be scaled by $10/R$ and the halving interval by $R/10$, keeping emission per second and total supply invariant.

Tail emission: None (pure deflationary after ~66 years)

16.2 Fee Structure

Minimum fee rate: 1 satoshi per byte (1 sat = 10^{-8} QTM)

Recommended fee: 10 sat/byte for normal priority

Fee calculation:

```
base_fee = tx_size_bytes × fee_rate
```

Minimum transaction fee (2-in/2-out, ~110 KB, see Section 9.4):

```
110,000 × 1 sat = 0.0011 QTM
```

16.3 Unit Definitions

1 QTM = 10^8 satoshi

Smallest unit: 1 satoshi = 10^{-8} QTM

Display formats:

QTM: Up to 8 decimal places

mQTM: Up to 5 decimal places (1 mQTM = 0.001 QTM)

sat: Integer only

Part III: Verification Criteria

17. Correctness Properties

17.1 Cryptographic Correctness

Property 1: Commitment Binding

For all PPT adversaries A :

$$\Pr[\text{VerifyOpening}(\text{pp}, c, v_1, r_1) \neq \text{VerifyOpening}(\text{pp}, c, v_2, r_2) \mid v_1 \neq v_2] < \text{negl}()$$

Property 2: Commitment Hiding

For all PPT adversaries A , all v_0, v_1 :

$$|\Pr[A(\text{Commit}(v_0)) = 1] - \Pr[A(\text{Commit}(v_1)) = 1]| < \text{negl}()$$

Property 3: STARK Soundness

For all PPT adversaries A :

$$\Pr[\text{Verify}() = \text{accept} \mid \text{statement is false}] < 2^{-100}$$

Property 4: STARK Zero-Knowledge

There exists simulator S such that:

$$\{\text{Prove}(\text{witness})\}_{\text{witness}} \approx_c \{S(\text{statement})\}_{\text{statement}}$$

17.2 Protocol Correctness

Property 5: Balance Preservation

For all valid transactions tx :

$$\Sigma(\text{input values}) = \Sigma(\text{output values}) + tx.\text{fee}$$

Property 6: No Double Spending

For all valid chains:

Each nullifier appears at most once

Property 7: Output Uniqueness

For all valid outputs in a chain:

Each (commitment, position) pair is unique

Property 8: Spend Authorization

Spending a note requires (a) a SPHINCS+ signature under the SpendPK whose hash is bound into the note commitment, and (b) knowledge of the nk preimage of the note's NullifierPK. Both are enforced by circuit constraints 7-8 (Section 10.1). In particular, the note's CREATOR cannot spend or re-nullify it: they know the value commitment's opening but neither SpendSK nor nk .

17.3 Privacy Properties

Property 9: Sender Privacy

Given a transaction tx, no PPT adversary can determine which outputs were spent with probability $> 1/N$ where N is the size of the anonymity set (entire output set)

Property 10: Receiver Privacy

Given a transaction tx, no PPT adversary can link outputs to recipient addresses without ViewSK

Property 11: Amount Privacy

Given a transaction tx, no PPT adversary can determine input or output values without corresponding keys

18. Test Vectors

18.1 Hash Function Test Vectors

STATUS: PLACEHOLDER. The concrete byte values below MUST be generated by the reference implementation before this specification is finalized. They are deliberately NOT given as literal hashes here - publishing invented values as "canonical" would cause every correct implementation to fail.

Test 1: H_nullifier

Domain tag: "Quantum-v1.nullifier"
 Input: $0x00 \times 64$ (64 zero bytes)
 Output: [TO BE COMPUTED - 32 bytes, SHAKE256 with domain prefix per Section 2.1]

Test 2: H_merkle leaf hash

Domain tag: "Quantum-v1.merkle"
 Input: $0x01 \parallel 0x00^{32}$ (prefix + 32 zero bytes)
 Output: [TO BE COMPUTED - 32 bytes]

Test 3: H_merkle node hash

Input: $0x00 \parallel \text{leaf1} \parallel \text{leaf2}$ (prefix + two 32-byte children)
 Where leaf1 = leaf2 = output from Test 2
 Output: [TO BE COMPUTED - 32 bytes]

Test 4: Domain separation verification

$H_{\text{nullifier}}(0x00^{32})$, $H_{\text{commitment}}(0x00^{32})$, $H_{\text{merkle}}(0x00^{32})$
 Required property: all three outputs are pairwise distinct
 (verifiable today from the construction; literal values to be computed)

18.2 Commitment Test Vectors

Test 5: Zero commitment

Input: $v = 0$, $r = \text{zero polynomial vector}$

Output: $c = A \cdot 0 + 0 \cdot e_1 = 0$ (zero vector in R_q^k)

Serialized: $0x00 \times 3072$ (all zero coefficients)

Test 6: Unit value commitment

Input: $v = 1$, $r = \text{zero polynomial vector}$

Output: $c = (1, 0, 0, 0)$ as constant polynomials

$c[0][0] = 1$, all other coefficients = 0

Test 7: Homomorphic property

Let r_1, r_2 be random polynomial vectors with coefficients in $[-2, 2]$

$\text{Commit}(100, r_1) + \text{Commit}(50, r_2) = \text{Commit}(150, r_1+r_2)$

Verification: Extract value limbs (Section 3.3), verify limb-wise

$100 + 50 = 150$; opening checked with relaxed bound = 2

Test 8: Binding test (negative)

Property: Cannot find $(v_1, r_1) \neq (v_2, r_2)$ such that $\text{Commit}(v_1, r_1) = \text{Commit}(v_2, r_2)$

Test: Generate 10^6 random commitments, verify no collisions

18.3 Transaction Test Vectors

Test 9: Minimal valid transaction (1-in, 1-out)

Input:

- Value: 1000000 satoshi (0.01 QTM)
- Position in tree: 0
- Nullifier key: $0x1a2b3c4d\dots$ (32 bytes)

Output:

- Value: 999999 satoshi
- Recipient: test address

Fee: 1 satoshi

Expected nullifier: $H_{\text{nullifier}}(\text{nk} || \text{commitment} || 0) = [\text{TO BE COMPUTED}]$

Balance check: $1000000 = 999999 + 1$

Serialized size: ~105 KB (1 input, 1 output)

Test 10: Standard transaction (2-in, 2-out)

Inputs: 500000 sat + 500000 sat = 1000000 sat

Outputs: 400000 sat + 590000 sat = 990000 sat

Fee: 10000 sat

Balance check: $1000000 = 990000 + 10000$

Serialized size: ~110 KB

Test 11: Maximum transaction (16-in, 16-out)

Maximum inputs: 16

Maximum outputs: 16

Serialized size: ~0.4 MB (outputs ~77 KB + proof, which grows with
16 in-circuit signature verifications)

Proof generation time: < 120 seconds (extended limit for max size)

18.4 Consensus Test Vectors

Test 12: Genesis block

See Appendix B for complete genesis block structure

Genesis hash: [TO BE COMPUTED at mainnet launch - see B.3]

First block (height 1) hash: [computed at launch]

Test 13: Difficulty adjustment example (DAG)

Given: 1000 blocks in epoch window

Target block rate: 10 blocks per second

Epoch duration: 100 seconds (expected 1000 blocks)

If actual blocks in epoch = 1200 (too fast):

New target = old_target / 1.2 (smaller target → harder mining)

If actual blocks in epoch = 800 (too slow):

New target = old_target / 0.8 (larger target → easier mining)

Adjustment ratio clamped to [0.75, 1.25] per epoch (DAG requires smoother adjustment)

Test 14: DAG block ordering (GhostDAG blue score)

Block A: blue_score = 1000, parents = [genesis]

Block B: blue_score = 1050, parents = [A, C]

Block C: blue_score = 1020, parents = [genesis]

Canonical ordering: Blocks ordered by blue_score (GhostDAG algorithm)

All valid blocks included in DAG (none orphaned)

19. Implementation Requirements

19.1 Mandatory Features

[MUST] Implement all cryptographic primitives from Part I

[MUST] Implement full transaction validation

[MUST] Implement STARK prover and verifier

[MUST] Implement full node with P2P networking

[MUST] Implement wallet with key management

[MUST] Pass all test vectors

[MUST] Achieve specified performance targets

19.2 Performance Targets

Proof generation: < 60 seconds per typical transaction (consumer CPU);

< 120 seconds for maximum-size (16-in/16-out) transactions

Proof verification: < 1 second per individual proof;

amortized < 10 ms per transaction via batch/aggregated verification

Block validation: < 10 seconds per block (1000 transactions, aggregated)

Wallet scanning: < 1 second per block

Merkle proof: < 10 ms

Nullifier lookup: < 1 ms

19.3 Security Requirements

[MUST] Use constant-time implementations for all secret operations

[MUST] Zeroize sensitive memory after use

[MUST] Validate all inputs before processing

[MUST] Implement rate limiting against DoS

[MUST] Use cryptographically secure random number generation

19.4 Code Quality Requirements

[MUST] Compile without warnings on strict settings

[MUST] Pass static analysis (clippy for Rust, etc.)

[MUST] Have >80% test coverage

[MUST] Document all public APIs

[MUST] Include fuzzing targets for parsers

20. Formal Verification Targets

20.1 Properties to Formally Verify

1. Type safety of all data structures
2. Memory safety (no buffer overflows, use-after-free)
3. Correctness of finite field arithmetic
4. Correctness of polynomial operations
5. Soundness of STARK verifier
6. Balance preservation in transaction validation
7. Nullifier uniqueness enforcement
8. Merkle tree correctness

20.2 Verification Tools

Recommended:

- Rust: MIRI for undefined behavior detection
- Rust: Kani for bounded model checking
- General: TLA+ for protocol logic
- Cryptographic: EasyCrypt for proof verification

Optional:

- Coq/Lean for full formal proofs
- F* for verified implementation extraction

20.3 Audit Checklist

- [] Cryptographic review by domain expert
 - [] Implementation review by security firm
 - [] Formal verification of critical paths
 - [] Fuzzing campaign (>1 billion iterations)
 - [] Incentivized testnet with bug bounty
 - [] Economic audit of incentive mechanisms
-

Part IV: Appendices

A. Reference Implementations

A.1 Polynomial Multiplication (NTT)

Two distinct NTTs exist in Quantum and must not be conflated: 1. The RING NTT mod $q = 8380417$ ($\zeta = 1753$) for lattice commitments. Because $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ is NEGACYCLIC, this NTT requires the standard pre/post twist by powers of ζ (as in CRYSTALS-Dilithium) — a plain cyclic NTT computes the wrong product. 2. The STARK field FFT over Goldilocks ($p = 2^{64} - 2^{32} + 1$) used by the prover for trace interpolation.

The skeleton below shows the butterfly structure only (cyclic form, generic modulus). Production code MUST add the negacyclic twist for case 1 and use constant-time modular reduction (Montgomery/Barrett), not the % operator.

```
type Felt = u64;
const P: u64 = 0xFFFFFFFF00000001; // modulus (Goldilocks shown; use q = 8380417 for R_

fn ntt(a: &mut [Felt; 256], omega: Felt) {
    let n = 256;
    let mut m = 1;
    while m < n {
```

```

    let w_m = pow_mod(omega, (n / (2 * m)) as u64);
    let mut k = 0;
    while k < n {
        let mut w = 1u64;
        for j in 0..m {
            let t = mul_mod(w, a[k + j + m]);
            let u = a[k + j];
            a[k + j] = add_mod(u, t);
            a[k + j + m] = sub_mod(u, t);
            w = mul_mod(w, w_m);
        }
        k += 2 * m;
    }
    m *= 2;
}
}

```

```

fn mul_mod(a: u64, b: u64) -> u64 {
    // Montgomery multiplication or Barrett reduction
    ((a as u128 * b as u128) % P as u128) as u64
}

```

A.2 Lattice Commitment

```

const N: usize = 256; // Polynomial degree
const K: usize = 4;    // Module rank
const Q: u32 = 8380417; // Modulus
const ETA: i32 = 2;    // Noise bound

type Poly = [i32; N];
type PolyVec = [Poly; K];

fn commit(a: &[PolyVec; K], v: u64, r: &PolyVec) -> PolyVec {
    let mut c = [[0i32; N]; K];

    // c = A · r
    for i in 0..K {
        for j in 0..K {
            let product = poly_mul(&a[i][j], &r[j]);
            for k in 0..N {
                c[i][k] = (c[i][k] + product[k]) % Q as i32;
            }
        }
    }
}

```

```

    // c[0] += Enc(v): 4 limbs of 16 bits in coefficients 0..3 (Section 3.3).
    // Never reduce v mod q directly - that silently wraps any amount
    // above ~0.08 QTM and breaks supply integrity.
    for limb in 0..4 {
        let v_limb = ((v >> (16 * limb)) & 0xFFFF) as i32;
        c[0][limb] = (c[0][limb] + v_limb) % Q as i32;
    }

    c
}

fn poly_mul(a: &Poly, b: &Poly) -> Poly {
    // NTT-based multiplication in R_q
    let mut a_ntt = ntt_forward(a);
    let b_ntt = ntt_forward(b);

    for i in 0..N {
        a_ntt[i] = ((a_ntt[i] as i64 * b_ntt[i] as i64) % Q as i64) as i32;
    }

    ntt_inverse(&a_ntt)
}

```

A.3 STARK Prover Outline

```

struct StarkProver {
    air: ArithmeticIntermediateRepresentation,
    fri_params: FriParameters,
}

impl StarkProver {
    fn prove(&self, witness: &Witness) -> StarkProof {
        // 1. Generate execution trace
        let trace = self.generate_trace(witness);

        // 2. Commit to trace polynomials
        let trace_polys = self.interpolate_trace(&trace);
        let trace_commitment = self.commit_polynomials(&trace_polys);

        // 3. Get challenge for constraint composition
        let alpha = self.fiat_shamir_challenge(&trace_commitment);

        // 4. Compute constraint composition polynomial
        let composition = self.compute_composition(&trace_polys, alpha);
        let composition_commitment = self.commit_polynomials(&[composition]);
    }
}

```

```
// 5. Get challenge for DEEP composition
let z = self.fiat_shamir_challenge(&composition_commitment);

// 6. Compute DEEP quotient
let deep_quotient = self.compute_deep_quotient(&trace_polys, &composition, z),

// 7. Run FRI protocol
let fri_proof = self.fri_prove(&deep_quotient);

// 8. Generate query responses
let queries = self.generate_queries(&trace_commitment, &fri_proof);

// 9. Proof of work grinding
let pow_nonce = self.grind_pow(&queries);

StarkProof {
    trace_commitment,
    composition_commitment,
    fri_proof,
    queries,
    pow_nonce,
}
```

B. Genesis Block

B.1 Genesis Parameters

[illegible]

Genesis message (encoded in first 80 bytes):

"Quantum Genesis - Quantum-Secure Privacy for All - 2026-01-01"

B.2 Genesis Block Structure

```
struct GenesisBlock {
    header: BlockHeader {
        version: 1,
        parent_hashes: vec![],           // No parents (DAG root)
        num_parents: 0,
        blue_score: 0,                   // Genesis has blue_score 0
        merkle_root: [0u8; 32],
        output_tree_root: [0u8; 32],
        nullifier_set_root: [0u8; 32],
        timestamp: 1767225600,           // Launch timestamp to be defined
        difficulty: GENESIS_DIFFICULTY,
        nonce: 0,
    },
    transactions: [],
}
```

```
// Genesis block is validated specially:
// - No PoW check (nonce = 0 is accepted)
// - No parent blocks required (DAG root)
// - Empty transaction list is valid
// - First mined block (height 1) follows normal rules
```

B.3 Genesis Block Hash

Genesis block hash (SHA3-256 of serialized header):

0x00000000000000000000000000000000[to be computed at mainnet launch]

Note: Testnet will use a different genesis block with timestamp of testnet launch.

C. Network Magic Numbers

Mainnet magic: 0x51544D31 ("QTM1" in ASCII)

Testnet magic: 0x51545430 ("QTT0" in ASCII)

Regtest magic: 0x51545230 ("QTR0" in ASCII)

Protocol version: 1

Minimum supported version: 1

D. Recommended Libraries

D.1 Rust Ecosystem

Cryptography:

- sha3: SHAKE256 implementation
- blake3: Fast hashing (non-consensus uses only)
- pqcrypto-kyber: ML-KEM implementation
- pqcrypto-sphincsplus: SPHINCS+ implementation

Note: elliptic-curve libraries (curve25519-dalek etc.) MUST NOT be dependencies - R2.1 prohibits EC cryptography anywhere in the system.

Proof systems:

- winterfell: STARK prover/verifier
- plonky2: Alternative STARK implementation

Networking:

- tokio: Async runtime
- snow: Noise protocol
- libp2p: P2P networking

Storage:

- rocksdb: Key-value store
- sled: Pure Rust alternative

D.2 Alternative Language Implementations

Go:

- gnark: ZK proof systems
- circl: Post-quantum crypto

C/C++:

- liboqs: Post-quantum algorithms
- libstark: STARK implementation

E. Glossary

AIR: Arithmetic Intermediate Representation

FRI: Fast Reed-Solomon Interactive Oracle Proof of Proximity

ML-KEM: Module Lattice Key Encapsulation Mechanism (Kyber)

NTT: Number Theoretic Transform

STARK: Scalable Transparent Argument of Knowledge

UTXO: Unspent Transaction Output

ZK: Zero-Knowledge

F. Document Metadata

Title: Quantum - Quantum-Secure Privacy Blockchain, Technical Specification
Version: 1.0
Status: Draft
Date: 2026-01-14
License: CC0 (Public Domain)

Authors: Phexora AI Research Team
Repository: <https://github.com/phexora/quantum>
Website: <https://quantum.phexora.ai>

Review status:

Cryptographic review: Pending (seeking external reviewers)
Implementation audit: Pending (no implementation yet)
Community feedback: Open for comments via GitHub issues

Document hash (SHA-256):

To be computed when document is finalized.
Use: `sha256sum zkprivacy-quantum-spec-v1.md`

G. Known Limitations and Trade-offs

This section documents known limitations and design trade-offs:

G.1 Transaction Size

Issue: Transactions are large (~110 KB for 2-in, 2-out)

Breakdown:

- STARK proof: ~100 KB (dominant factor; SPHINCS+ signatures are witness data verified in-circuit and never appear on-chain)
- Lattice commitments: ~3 KB per output
- Kyber ciphertext: ~1.5 KB per output
- Note commitment + encrypted data: ~0.2 KB per output

Impact with DAG (10-32 blocks/second):

- Higher bandwidth requirements (~110 MB/sec at 1000 TPS sustained)
- Larger blockchain storage (~3.5 PB/year archival at 1000 TPS)
- Requires high-bandwidth nodes for full validation (1 Gbps, R6.2)

DAG scaling:

- At 10 blocks/sec with 100 tx/block = 1,000 TPS (~110 MB/sec, ~11 MB/block)
- Parallel blocks distribute load across network
- Light clients only validate headers + proofs

Mitigation:

- Proof aggregation for blocks (REQUIRED at target throughput, R6.1)
- Recursive STARKs for smaller proofs (research area)
- Accept bandwidth trade-off for quantum security + privacy

G.2 Proof Generation Time

Issue: Proof generation takes 30-120 seconds

Cause: STARK provers are computationally intensive

Impact:

- User experience: waiting time for transaction confirmation
- Mobile devices may struggle

Mitigation:

- Hardware acceleration (GPU/FPGA)
- Incremental proving during wallet sync
- Pre-computation of partial proofs
- Accept trade-off for transparency (no trusted setup)

G.3 Address Size

Issue: Addresses are large (1,664 bytes)

Cause:

- SPHINCS+ public key: 64 bytes
- ML-KEM-1024 public key: 1,568 bytes
- Nullifier public key: 32 bytes

Impact:

- Cannot use short addresses for display
- QR codes are large

Mitigation:

- Use shortened address (32-byte hash) for display/verification
- Full address only needed in transaction data

G.4 No Smart Contracts

Issue: Version 1.0 does not support programmable transactions

Reason: Complexity and attack surface reduction

Future work:

- Version 2.0 may add ZK-compatible smart contracts
 - Research into STARKs for general computation
-

H. Research Challenges and Future Directions

This section documents the core research challenges that define Quantum, as well as potential future improvements.

H.1 Core Research Challenge: Privacy-Preserving DAG Consensus

This is the defining research contribution of Quantum. No existing blockchain combines DAG-based consensus with full transaction privacy. Solving this enables a system that is simultaneously:

- Fast (1,000+ TPS via parallel blocks) - Private (full anonymity set, hidden amounts) - Quantum-secure (post-quantum cryptography throughout)

The problem statement:

- Given: GhostDAG consensus (parallel block creation, no orphans)
- Given: STARK-based transaction proofs (quantum-secure, no trusted setup)
- Given: Privacy requirements (hidden amounts, unlinkable outputs)

Design: A protocol where privacy guarantees hold despite parallel block ordering

Open Research Problems (must be solved for conformant implementation):

1. Parallel Nullifier Commitment

Problem: Nullifiers prevent double-spend, but two parallel blocks might both try to spend the same output.

Approaches under investigation:

- Nullifier epochs: Commit nullifiers in batches at DAG "checkpoints"
- Optimistic execution: Accept parallel spends, resolve in ordering phase
- DAG-aware nullifier sets: Nullifier validity depends on DAG ancestry

Required property: No double-spend possible, even with adversarial miners

2. Anonymity Set Coherence

Problem: "All outputs" as anonymity set assumes linear history.
DAG has multiple valid topological orderings.

Approaches under investigation:

- Canonical ordering: Define deterministic topological sort of DAG
- Epoch-based sets: Anonymity set = outputs confirmed before epoch boundary
- Branch-aware proofs: Prove membership in "any valid ordering"

Required property: Anonymity set size never smaller than sequential chain

3. STARK Proofs Over DAG Structure

Problem: Proving transaction validity requires proving output membership.

DAG membership is more complex than Merkle tree membership.

Approaches under investigation:

- DAG Merkle structures: Generalized authenticated data structures for DAGs
- Epoch snapshots: Prove against periodic linear snapshots
- Recursive proofs: Prove validity relative to parent blocks' proofs

Required property: Proof size and verification time remain practical

4. Transaction Graph Privacy

Problem: DAG's explicit parent-child structure might enable taint analysis even when amounts and addresses are hidden.

Approaches under investigation:

- Decoy references: Transactions reference random additional parents
- Delayed inclusion: Random delay before transaction enters DAG
- Reference obfuscation: Parents selected to minimize information leakage

Required property: DAG structure reveals no more than sequential chain

5. In-Circuit Post-Quantum Spend Authorization

Problem: Spend authorization verifies SPHINCS+ signatures INSIDE the STARK (Section 10.1, constraint 8) because revealing signature keys on-chain would link spends by the same wallet - and unlike elliptic-curve schemes, hash-based keys cannot be re-randomized into sender-derivable one-time keys (the PQ stealth-address problem). SPHINCS+ verification is pure hashing and therefore STARK-compatible, but it requires thousands of hash evaluations per input, dominating prover cost.

Approaches under investigation:

- Optimized SPHINCS+ parameter sets for in-circuit verification
- Hash-preimage spend authorization (prove knowledge of SpendSeed instead of verifying a signature; cheaper, but loses the key separation useful for hardware wallets)

- STARK-friendly hash choices (e.g. Poseidon2/Rescue over Goldilocks) for the note-binding and nullifier constraints

Required property: Proof generation stays within R6.1 limits with spend authorization fully enforced in zero-knowledge

Research Milestones:

- M1: Formal security model for privacy in DAG consensus
- M2: Nullifier scheme with proof of double-spend prevention
- M3: STARK circuit for DAG membership proofs
- M4: Privacy proof under DAG adversary model
- M5: Reference implementation passing all security tests
- M6: Independent security audit

H.2 Alternative Scaling Approaches Considered

Before selecting GhostDAG, several alternative scaling approaches were evaluated:

Ouroboros Leios (Cardano) [Ouroboros Leios](#) achieves ~1,000 TPS through pipelining with three block types:

Architecture:

- Input Blocks (IBs) → Collect user transactions
- Endorser Blocks (EBs) → Committee validates transactions
- Ranking Blocks (RBs) → Finalize ordering (20-second intervals)

Results: 30-50x throughput improvement over base Praos

Why not adopted:

1. Proof of Stake conflicts with privacy
 - Stake is visible on-chain, compromising holder privacy
 - Stake-weighted selection reveals economic information
 - Committee membership leaks participation data
2. Committee-based endorsement adds trust assumptions
 - Requires honest committee majority
 - Committee selection must be unpredictable
 - Conflicts with Quantum's trustless design goal
3. Complexity
 - Three block types vs. GhostDAG's uniform blocks
 - More attack surface for privacy analysis

Sharding (Ethereum-style) Sharding splits the network into parallel chains processing different transactions.

Why not adopted:

1. Cross-shard privacy is unsolved
 - Transactions crossing shards leak information
 - Anonymity set fragments across shards
 - Atomic cross-shard private transactions are an open problem
2. Shard assignment reveals information
 - Which shard holds your outputs?
 - Shard-specific scanning leaks user locations
3. Complexity
 - Shard coordination protocols
 - Data availability across shards
 - Reorganization handling across shards

Parallel Execution (Solana-style) Solana achieves high throughput through parallel transaction execution with declared state access.

Why not adopted:

1. State access declarations leak privacy
 - Transactions must declare which accounts they touch
 - This reveals transaction graph information
 - Conflicts with unlinkability requirements
2. Hardware requirements harm decentralization
 - Solana requires high-end specialized hardware
 - Validator sets are permissioned in practice
3. Centralization pressure
 - Validator hardware is specialized (not commodity)
 - Small validator set compared to PoW miner distribution

GhostDAG (Selected) Why GhostDAG fits Quantum:

1. Pure Proof of Work
 - No stake visibility (privacy preserved)
 - No committees (trustless)
 - Permissionless mining (anyone can participate)
 - High hashrate = high security
2. Uniform block structure
 - All blocks are equal (no special roles)
 - Simpler privacy analysis
 - Cleaner anonymity set definition

3. Proven in production

- Kaspa demonstrates 10+ blocks/second
- Well-understood security properties
- Active research community

4. Privacy-compatible (with research)

- Parallel blocks don't inherently leak more than sequential
- Research challenges are tractable (see H.1)
- No fundamental conflicts with privacy requirements

Summary:

Approach	Throughput	Privacy Compatible	Trust Model	Selected
Leios (PoS)	~1,000 TPS	No (stake visible)	Committee	No
Sharding	~10,000 TPS	No (cross-shard leaks)	Per-shard	No
Solana-style	~65,000 TPS	No (state declarations)	Validators	No
GhostDAG (PoW)	~1,000+ TPS	Yes (with research)	Pure PoW	Yes

H.3 Mining Algorithm Analysis (Open Research)

The mining algorithm choice is an **open research question** for DAG-based privacy blockchains. The focus is on **security through high hashrate** and **permissionless participation**, not ASIC resistance.

Key Insight: Bitcoin and Kaspa prove that ASIC/GPU mining does not harm decentralization. What matters is: - Permissionless participation (anyone can mine) - Geographic distribution (no single point of failure) - High total hashrate (expensive to attack)

DAG-Specific Requirements: 1. **Fast verification:** 10-32 blocks/second means verifying 320+ block headers per 10-second window 2. **Low latency:** Block propagation must be fast to prevent DAG fragmentation 3. **High hashrate:** More hashrate = more security against 51% attacks 4. **Post-quantum consideration:** Hash-based PoW is inherently quantum-resistant

Algorithm Comparison:

	kHeavyHash	SHA-256d	RandomX
Hardware	GPU/ASIC	ASIC	CPU
Verification	<0.1ms/hash	<0.01ms/hash	~2-3ms/hash
Hashrate security	High	Highest	Lower
Permissionless	Yes	Yes	Yes
Production use	Kaspa (2+ yr)	Bitcoin (15 yr)	Monero (5+ yr)
DAG suitability	Proven	Proven (fast)	Uncertain

Analysis:

1. **kHeavyHash**: Proven for DAG consensus (Kaspa). High hashrate with GPU/ASIC mining. Maintains permissionless participation. **Recommended starting point** based on Kaspa's success.
2. **SHA-256d**: Highest hashrate security (Bitcoin). Extremely fast verification suits high block rates. ASICs provide maximum security investment. Worth considering for maximum security.
3. **RandomX**: CPU-only limits total hashrate and thus security. Verification speed may bottleneck DAG. Not recommended for high-throughput DAG.

Research Required: - Analyze kHeavyHash hashrate growth and miner distribution - Evaluate SHA-256d for DAG compatibility (verification throughput) - Consider merged mining possibilities for bootstrap security

Current Position: **kHeavyHash is the leading candidate** based on Kaspa's proven DAG performance. SHA-256d is a viable alternative if maximum hashrate security is prioritized. Final decision in implementation phase.

H.4 Proof System Improvements

Current: STARKs with ~100KB proofs, 30-120 second generation

Research Directions:

1. Recursive STARKs
 - Prove verification of other proofs
 - Enables proof aggregation: one proof for entire block
 - Could reduce per-transaction overhead by 10-100x
2. Hardware Acceleration
 - GPU-based STARK provers
 - FPGA implementations for mobile/embedded
 - Could reduce proof generation to <10 seconds
3. Folding Schemes (Nova, SuperNova)
 - Incrementally verifiable computation
 - Could enable real-time proof updates
 - Active research area, not yet production-ready
4. Hybrid STARK-SNARK
 - Use STARKs for quantum security
 - Wrap in SNARK for smaller on-chain footprint
 - Maintains no-trusted-setup property
5. Nullifier Set Accumulators
 - The nullifier set grows unboundedly (~2 TB/year at target throughput) and cannot be pruned (see R6.2)

- Hash-based accumulators with non-membership proofs could bound node storage at the cost of larger transactions

H.5 Layer 2 Scaling

Rather than modifying L1 consensus, scaling could be achieved via Layer 2:

Payment Channels:

- Off-chain transactions between parties
- Only settlement on L1
- Challenge: Privacy-preserving channel design

Rollups:

- Batch transactions off-chain
- Single validity proof on L1
- Challenge: Maintaining privacy across rollup boundary

State Channels:

- Generalized off-chain state updates
- Requires ZK-compatible state transition proofs

Note: L2 solutions are complementary to, not replacements for, L1 improvements. Both can be pursued independently.

H.6 Post-Quantum Signature Alternatives

Current: SPHINCS+-256f (49KB signatures)

Potential Alternatives:

FALCON (NIST standardized):

- Signature size: ~1.3 KB (40x smaller)
- Trade-off: Requires careful implementation (floating point)
- Risk: More complex, potential side-channel vulnerabilities

SLH-DSA (SPHINCS+ successor):

- NIST's final standard (2024)
- Similar to SPHINCS+ with minor improvements
- Migration path: Parameter update, not architectural change

Dilithium (ML-DSA):

- Signature size: ~2.4 KB
- Lattice-based (same assumption family as commitments)
- Trade-off: Larger than FALCON, simpler than SPHINCS+

v1.0 Rationale: SPHINCS+ is the most conservative choice—pure hash-based security with no algebraic structure to attack. Size is acceptable for a chain prioritizing security over throughput. Future

versions may offer signature algorithm flexibility if smaller alternatives prove equally secure in production.

End of Specification

This document contains all information necessary to implement a complete, quantum-secure, privacy-by-default blockchain. Implementations **MUST** conform to all requirements marked [MUST] and **SHOULD** implement all performance optimizations.

Any ambiguity in this specification should be resolved by reference to the stated security properties and the principle of conservative security (when in doubt, choose the more secure option).